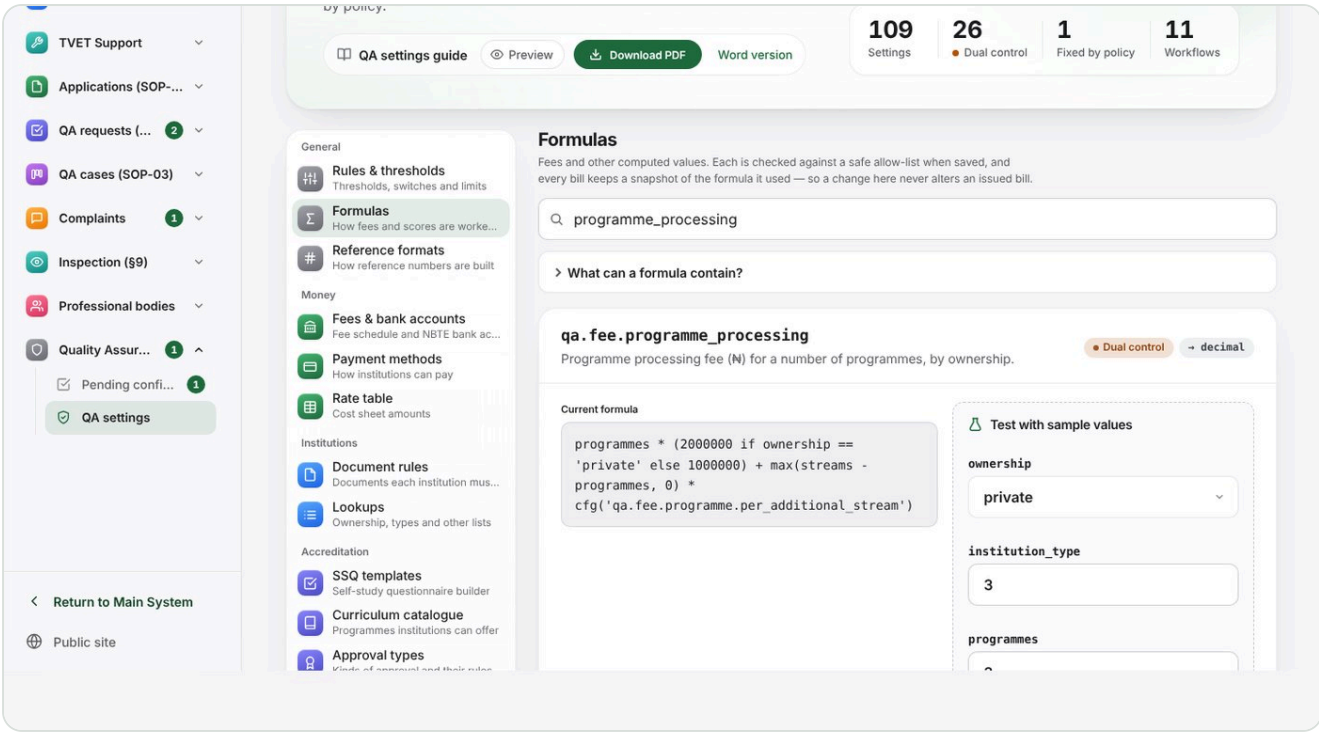


NBTE Digital QA System
QA Settings Guide

Every section of QA settings explained in plain English and technically: what it controls, who may change it, worked examples, formulas, workflows and troubleshooting.

National Board for Technical Education
NBTE ICT Directorate
Edition 1 · 21 October 2026

Contents

NBTE QA Settings Guide

1. Before you start
2. How a change flows through the system
3. The settings sections, one by one
4. Pending confirmations
5. The e-mail outbox
6. Walkthroughs with worked examples
7. Testing your change safely
8. The most important settings keys
9. Common mistakes
10. Troubleshooting

Appendix A. Addresses at a glance

Appendix B. Where things live in the code

NBTE QA Settings Guide

This guide is for the people who configure the NBTE Digital QA System: **QA settings administrators**, who make changes, and **Directors**, who confirm the changes that need a second officer. It explains every section of **QA settings**, what each one controls, who may change it, how a change takes effect and what else in the system it touches. It then walks through the most common changes step by step, with worked examples, and ends with a list of the most important settings, common mistakes and a troubleshooting table.

You do not need to be a programmer to use this guide. Each feature is described twice:

- **In plain English.** What it is, why it matters and what you see on screen.
- **Technically.** The names used in the code (setting keys, permissions, web addresses, database records), so that ICT staff and developers can find the same thing behind the screen.

Everything in this guide has been checked against the system's source code (release *Phase 8*). Where the screen and an earlier document disagree, the code wins and this guide says so.

Words used in this guide.

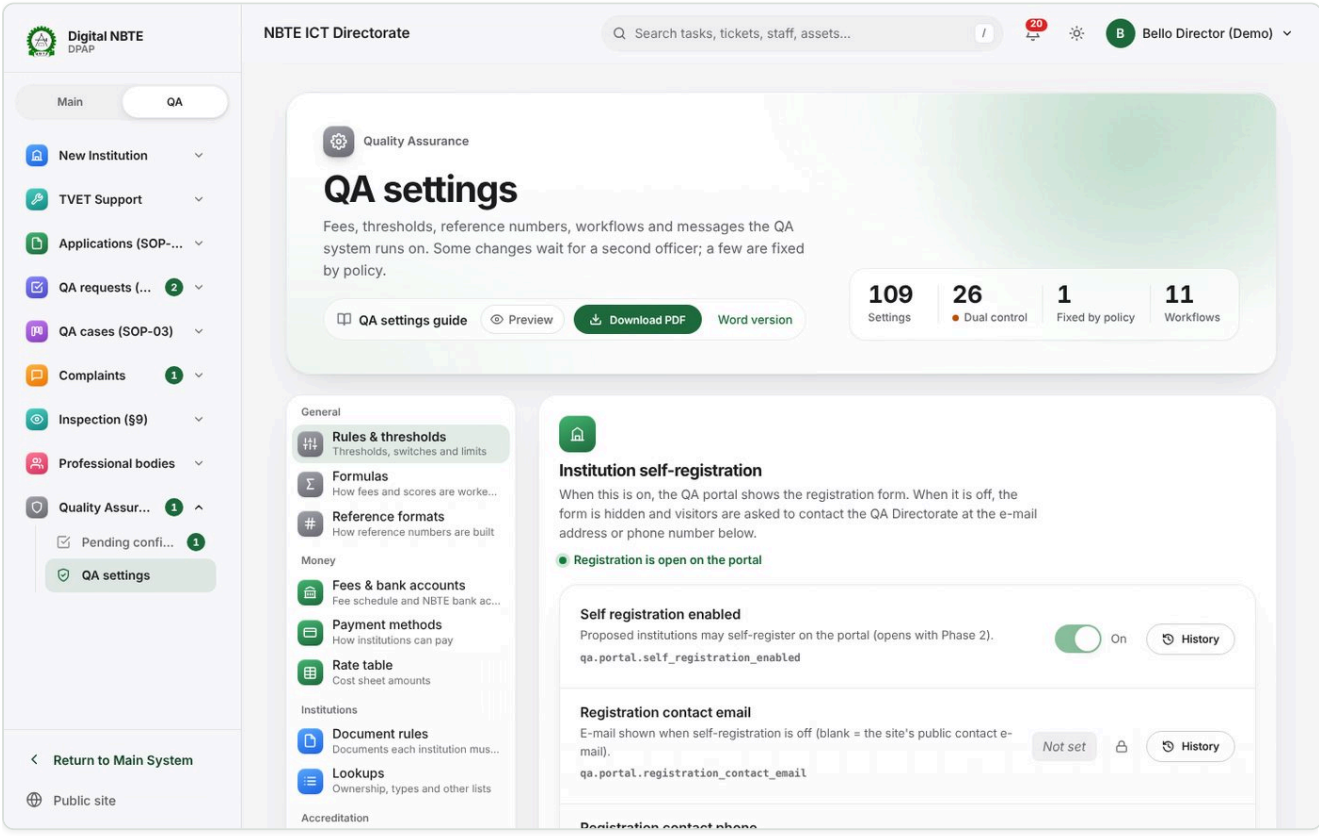
Word	Meaning
Setting	One named value the system reads, such as the number of days an institution has to answer a query. Each has a **key** such as `qa.query.default_due_days`
Formula	A setting whose value is a short calculation, such as a fee, written in a small, safe formula language
Table	A list of rows edited on its own page: fee schedules, rates, approval types, document rules, complaint routing rules and so on
Direct	Takes effect as soon as you save
Dual control	Saving creates a **pending confirmation**; nothing changes until a second officer confirms it
Fixed	Fixed by policy; shown for information and cannot be changed on screen
Pending confirmation	A proposed change waiting for a second officer (reference `PA-YYYY-NNNNNN`)
Snapshot	A frozen copy of the values used at the time, kept on a bill, an assessment, a submission or a cost line so a later settings change never alters it
Training copy	A copy of the system with demonstration data, used to try a change before making it on the live system

1. Before you start

1.1 Opening QA settings

In plain English. QA settings is part of the internal NBTE system (the NBTE address, not the institutions' portal). Switch the sidebar to the **QA** workspace and open **QA settings**, or go straight to `/qa/settings/`, which opens the first section, **Rules & thresholds**.

The sections are listed down the left side in five groups. Each has a coloured tile and one line saying what it is for. On a phone or tablet the list becomes a strip across the top that you can swipe.



QA settings, Rules and thresholds: the grouped navigation on the left (General, Money, Institutions, Accreditation, Cases & messages), the header with the QA settings guide, and the rules grouped by subject with Direct, Dual control and Fixed badges

Group	Section (tab)	Address	What it controls, in one line
General	Rules & thresholds	`/qa/settings/rules/`	Thresholds, switches and limits
General	Formulas	`/qa/settings/formulas/`	How fees and scores are worked out
General	Reference formats	`/qa/settings/references/`	How reference numbers are built

Group	Section (tab)	Address	What it controls, in one line
Money	Fees & bank accounts	`/qa/settings/fees/`	Fee schedule and NBTE bank accounts
Money	Payment methods	`/qa/settings/payment-methods/`	How institutions can pay
Money	Rate table	`/qa/settings/rates/`	Cost sheet amounts
Institutions	Document rules	`/qa/settings/documents/`	Documents each institution must hold
Institutions	Lookups	`/qa/settings/lookups/`	Ownership, types and other lists
Accreditation	SSQ templates	`/qa/settings/ssq-templates/`	Self-study questionnaire builder
Accreditation	Curriculum catalogue	`/qa/settings/catalogue/`	Programmes' required facilities and equipment
Accreditation	Approval types	`/qa/settings/approval-types/`	Kinds of approval and their rules
Accreditation	Workload limits	`/qa/settings/workload/`	Fair duty limits per rank
Cases & messages	Complaints	`/qa/settings/complaints/`	Layers, routing and escalation
Cases & messages	Workflows	`/qa/settings/workflows/`	Stages each process moves through
Cases & messages	Notification templates	`/qa/settings/templates/`	Words sent in-app and by e-mail

Two related pages sit outside the tab list but belong to the same job: **Pending confirmations** (`/qa/pending/`, section 4) and the **e-mail outbox** (`/qa/outbox/`, section 5).

Technically. The tab list is `TABS` and `TAB_META` in `qa_core/views.py`. Rules, formulas, reference formats, workflows, notification templates, pending confirmations and the outbox are in the `qa` URL namespace (`qa_core/urls.py`); the other tabs are in the `qa_set` namespace (`qa_core/settings_urls.py`), both mounted under `/qa/`.

1.2 Who can open QA settings

In plain English. Only NBTE staff accounts can open QA settings, and only if they hold at least one settings permission. Being able to *open* the pages is not the same as being able to *change* them: most people who open QA settings can read every section but change only the ones their permissions allow. A section you cannot change shows its values read-only, with a note such as *You cannot edit this: Needs qa_core.edit_qa_settings*.

One exception: **Finance officers** who hold only the Finance permissions can open **Fees & bank accounts** and **Payment methods** (to propose fees and bank details), but not the other sections.

Technically. `qa_core.permissions.can_view_qa_settings(user)` is true for a staff user who is a superuser or holds any of: `qa_core.edit_qa_settings`, `qa_core.edit_qa_dual_control_settings`, `qa_core.manage_flows`, `qa_core.publish_flow_version`, `qa_core.manage_reference_formats`, `qa_core.manage_notification_templates`, `accreditation.manage_ssq_templates`, `accreditation.publish_ssq_template`, `accreditation.manage_catalogue`, `accreditation.manage_rate_table`, `complaints.manage_complaint_layers`, `complaints.manage_routing_rules`, `complaints.manage_escalation_rules`, `complaints.manage_knowledge_base`. The Fees and Payment methods tabs use `payments.views._can_see_fee_settings`, which also admits `payments.view_obligation` and `payments.manage_bank_accounts`. Anyone else gets *403 Forbidden*.

1.3 Three kinds of control

In plain English. Every setting on **Rules & thresholds** and **Formulas** carries one of three badges:

Badge	What happens when you save	Examples
Direct	The new value applies at once. The change is recorded in the setting's **History**	Days to answer a query, reminder percentages, portal self-registration
Dual control (*Needs a second officer*)	Nothing changes yet. A pending confirmation is created and a second officer must confirm it	Every fee formula and fee amount, payment rules, impersonation limits, the capacity formula, the letter signatory
Fixed (*Fixed by policy*)	There is no edit button. Changing it needs a code release	`qa.payment.provider_confirmation_is_evidence_only` (a gateway report is evidence, never confirmation)

The other sections are **tables** (fee schedules, rates, approval types and so on). Some of their changes are dual control too (bank accounts, fee schedules, new fees, reference formats); the rest save directly and are recorded in the audit log.

Technically. QA settings are `config.ConfigEntry` rows whose key starts with `qa.` and whose scope is `global`. Each row carries `dual_control`, `editable_by` (the permission needed to edit it; blank means `qa_core.edit_qa_settings`) and the lock (`editable_by = "config.locked_by_policy"` or a key in `config.models.LOCKED_KEYS`). The write path is `qa_core/settings_service.py`: `can_edit()` checks the permission, `save_setting()` either saves (direct) or calls `actions.initiate("config.update_setting", …)` (dual control). Nobody can change a setting while impersonating another user.

1.4 Who may change what

In plain English. The table shows each permission, what it unlocks and who holds it after a standard installation (`seed_startup`). ICT can give a permission to more people by adding them to a group or changing their role. Note that the **QA Settings Admin role** and the **QA Settings Admin group** are slightly different bundles; the table names both.

Permission	What it unlocks	Seeded holders
`qa_core.edit_qa_settings`	Edit rules, formulas (propose, for dual-control ones), workload limits	QA Settings Admin role and group
`qa_core.edit_qa_dual_control_settings`	**Confirm** dual-control setting changes and reference-format changes	Director of Department role; Director of Finance (QA) role; *Finance – Approvals* group
`qa_core.manage_reference_formats`	Propose a reference-format change	QA Settings Admin role and group
`qa_core.manage_notification_templates`	Edit notification templates and switch them on or off	QA Settings Admin role and group
`qa_core.manage_flows`	Reserved for editing workflow drafts (no on-screen editor in this release, see 3.14)	QA Settings Admin role and group
`qa_core.publish_flow_version`	Reserved for publishing a workflow version (see 3.14)	Director of Department role
`qa_core.confirm_any_department`	Confirm pending items of any department	Executive Secretary role; *QA Senior Confirmers* group
`payments.view_obligation`	Propose a new fee, a fee-schedule change, a bank account, a bill by hand	QA Finance Officer and Director of Finance (QA) roles; *Finance – Payment Verification* group. The Executive Secretary and ICT QA Monitor roles also hold it (to read bills), which technically lets them propose too
`payments.manage_fee_schedule`	**Confirm** a new fee or a fee-schedule change	Director of Finance (QA) role
`payments.manage_bank_accounts`	**Confirm** a bank-account change	Director of Finance (QA) role
`payments.final_confirm_payment`	**Confirm** a bill issued by hand (and every payment)	Director of Finance (QA) role; *Finance – Approvals* group
`accreditation.manage_rate_table`	Edit the rate table	Director of Finance (QA) role; QA Settings Admin group
`institutions.manage_document_requirements`	Edit document rules	QA Settings Admin role

Permission	What it unlocks	Seeded holders
`institutions.manage_institution_lookups`	Edit lookups	QA Settings Admin role
`accreditation.manage_ssq_templates`	Start and edit SSQ template drafts	Director, Curriculum Officer roles; QA Settings Admin group
`accreditation.publish_ssq_template`	Publish an SSQ template version	Director of Department role
`accreditation.manage_catalogue`	Edit the curriculum catalogue	Director, Curriculum Officer roles; QA Settings Admin group
`accreditation.manage_approval_types`	Edit approval types	QA Settings Admin role and group
`complaints.manage_complaint_layers`	Edit complaint layers	QA Settings Admin role; *Complaints Admin* group
`complaints.manage_routing_rules`	Edit routing rules (and the setting `qa.complaints.fallback_department`)	QA Settings Admin role; *Complaints Admin* group
`complaints.manage_escalation_rules`	Edit escalation rules	QA Settings Admin role; *Complaints Admin* group
`complaints.manage_knowledge_base`	Edit help articles	QA Settings Admin role; *Complaints Admin* group

The QA Settings Admin role has rank 0, so a settings administrator can never confirm a dual-control change: that is the point of the control. A Director can confirm but, unless also given `qa_core.edit_qa_settings`, cannot start a settings change.

Technically. Role permissions are in `qa_core/seed_data.py` (`ROLE_PERMISSIONS`, `GROUPS`) and the phase files `seed_data_phase2.py` to `seed_data_phase8.py` (additive). Role ranks: Executive Secretary 999, Director 100, Director of Finance (QA) 100, Deputy Director 80, Division Head 50, Officer 20, QA Settings Admin 0.

1.5 The rules of dual control

In plain English. When a change needs a second officer, four rules always apply, and nobody (not even a superuser) can switch them off:

1. **You can never confirm your own change.**
2. **The confirmer must hold the confirm permission and be senior enough** (rank 50 or more for settings and reference formats; rank 100 or more for fees, bank accounts and bills).
3. **The confirmer must normally be in the same department as the officer who started it.** Officers with *confirm any department* (the Executive Secretary, the *QA Senior Confirmers* group) are the exception.

4. **A change is never applied over newer data.** If the record changed after the request was started, confirming fails safely with *Stale — the item changed after this was started. Please re-initiate.*

Only one pending change can be open on the same item at a time. A pending change that nobody decides **expires** (after 72 hours by default) and nothing is applied.

In practice this means: a settings change started by a QA settings administrator in DPAP is confirmed by the **Director of DPAP**; a new fee started by a Finance officer is confirmed by the **Director of Finance**. The Director of Finance cannot confirm a fee-formula change started in DPAP unless given *confirm any department*.

Technically. Policies are `qa_core.ActionPolicy` rows:

Action code	Label	Started with	Confirmed with	Min. rank	Same department
`config.update_setting`	Change a dual-control QA setting	the row's `editable_by`	`qa_core.edit_qa_dual_control_settings`	50	Yes (initiator's department)
`qa_core.update_reference_format`	Change a reference format	`qa_core.manage_reference_formats`	`qa_core.edit_qa_dual_control_settings`	50	Yes
`payments.update_fee_schedule`	Change Fee Schedule	`payments.view_obligation`	`payments.manage_fee_schedule`	100	Yes
`payments.create_fee_schedule`	Add Fee Schedule	`payments.view_obligation`	`payments.manage_fee_schedule`	100	Yes
`payments.update_bank_account`	Change Bank Account	`payments.view_obligation`	`payments.manage_bank_accounts`	100	Yes
`payments.issue_bill`	Issue a Bill	`payments.view_obligation`	`payments.final_confirm_payment`	100	Yes

The rules are enforced by `qa_core.actions.can_decide()`. Expiry is the policy's `expires_after_hours` or, when blank, `qa.settings.pending_action_expiry_hours`. These policies are data but have no edit screen (the Django admin shows them read-only).

1.6 Use a training copy first

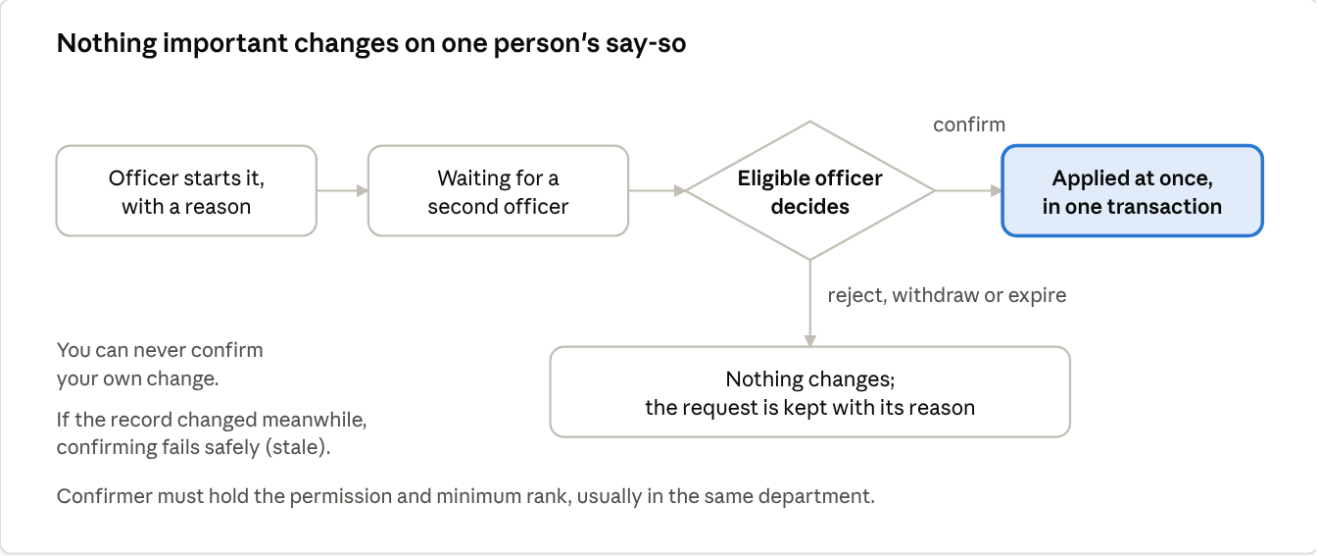
In plain English. Every change in this guide can be tried first on a **training copy**: a separate copy of the system with demonstration institutions and accounts, where nothing reaches a real institution. ICT can start one alongside the live system (Part A of the master guide, *Trying it without touching real data*; Part E, *Demo accounts for training*). Section 7 explains how to test safely.

---

2. How a change flows through the system

In plain English. Every settings change follows the same path:

1. **You propose it.** You open the section, change a value and save, giving a reason where asked.
2. **It is checked.** The system refuses a value of the wrong type (text where a number is needed, JSON that does not read, a formula that uses an unknown name), a value identical to the current one, or a change you are not allowed to make. You see the reason on the form.
3. **Direct changes apply at once. Dual-control changes wait.** A dual-control change becomes a pending confirmation. Eligible confirmers are told in the system and by e-mail; you see *Sent for confirmation — PA-2026-000123*. The old value stays in force.
4. **A second officer decides.** On **Pending confirmations** the confirmer sees the current value next to the proposed one, and confirms or rejects (a reason is required to reject). You are told the outcome.
5. **The new value is used from the next time the system reads it.** Settings are read when they are needed, so the change affects the *next* bill, the *next* task, the *next* reminder, the *next* complaint routed, and so on.
6. **Work already done keeps the old values.** Bills, payment slips, capacity assessments, SSQ submissions, priced cost lines, issued references, programme expiry dates and running workflows store what they used at the time. A later change never rewrites them.
7. **Everything is recorded.** Each change is written to the audit log with who, when and the before and after values. **History** on each rule shows the last 20 changes and the pending items for that rule.



Two-step control: one officer starts the change, it waits, a second officer decides, and only then is it applied

The table shows, for each kind of change, what is affected and what is not.

You change…	It affects	It does not affect
A fee formula or fee amount	Bills issued after the change; the live estimate on the portal	Bills already issued (the formula, inputs and amount are frozen on the bill)
A deadline (SLA hours, confirmation window, query days)	Tasks, queue items and queries opened after the change	Due dates already set on open tasks and items
Reminder percentages	The next reminder sweep, including tasks already open	Reminders already sent
A reference format	References issued after the change	References already issued (stored text)
The rate table	Cost lines added after the change	Cost lines already priced
The curriculum catalogue	SSQs opened or filled after the change	SSQs already submitted (their snapshot keeps the old items and quantities)
An SSQ template	SSQs opened after the new version is published	SSQs already opened (they stay on their version)
An approval type's duration	Decisions applied to programmes (stage 17) after the change	Programmes whose decision was already applied (their expiry date is stored)
Workload limits	The next assignment, booking or finder search	Assignments already made
A capacity constant or formula	Capacity assessments run after the change	Stored assessments (they keep a snapshot of every constant and formula)
Complaint routing or escalation	The next stage a complaint enters; the next SLA sweep	The handler already assigned to the current stage
A notification template	Messages sent after you save	Messages already sent
Portal self-registration	The portal's pages and register form at once	Institutions already registered
A workflow	New records, once a new version is published	Records already running (pinned to their version)

Technically. `config.get()` reads through a cache that is invalidated whenever a `ConfigEntry` is saved (`config/signals.py`), so with the production Redis cache a change is visible to every process at once. (With the development in-memory cache, other processes such as the Celery worker may see the old value for up to `CONFIG_CACHE_TTL` = 300 seconds.) Snapshots: `PaymentObligation.formula_snapshot` (`qa_core.formula.evaluate_setting`), `CapacityAssessment.parameters_snapshot`, the SSQ `Submission` snapshot (`accreditation.ssq.render_snapshot`), `CostLine.amount`, stored reference strings,

`Programme.expiry_date`, `FlowInstance.flow_version`. Every audited model writes a `core.AuditLogEntry`.

---

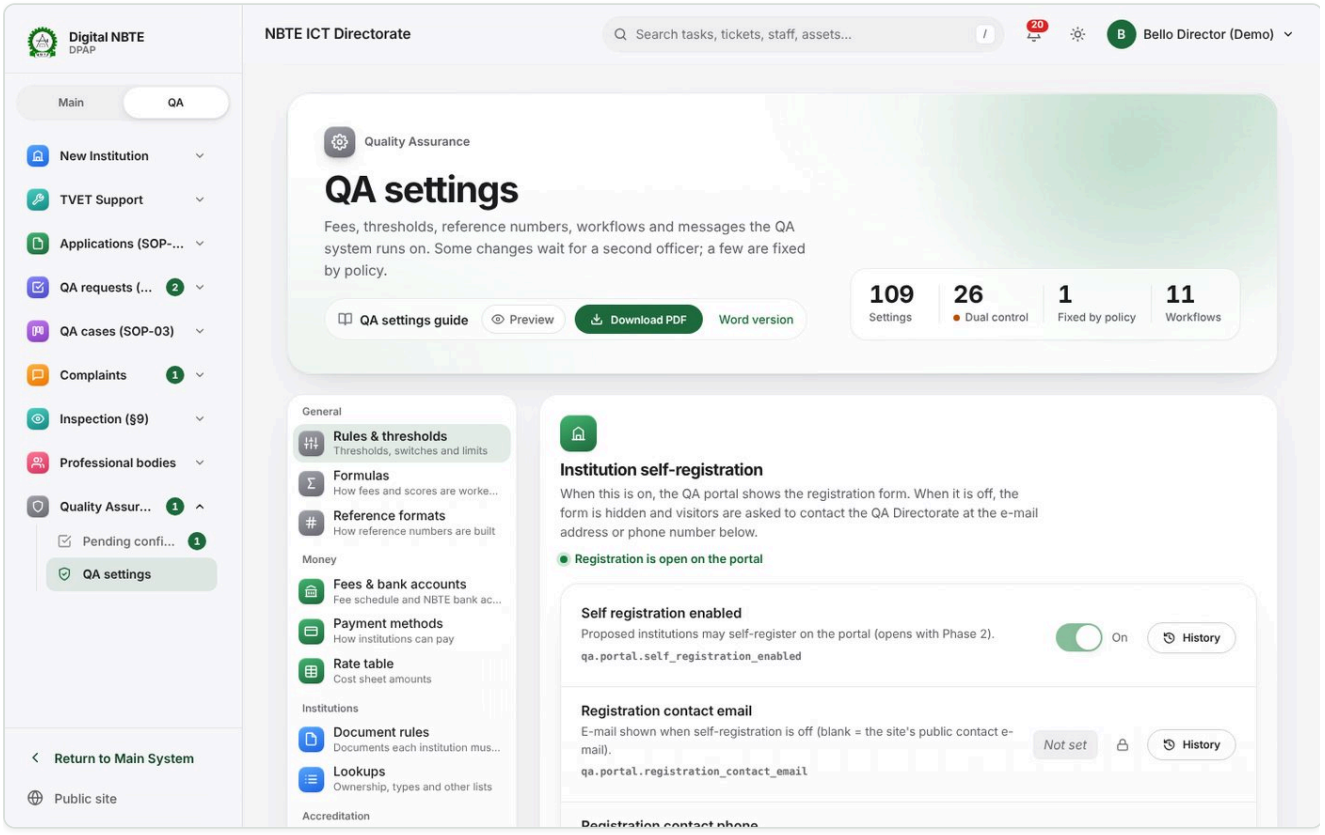
3. The settings sections, one by one

Each section below follows the same pattern: what it controls, who can change it, how a change takes effect and what it does to the rest of the system.

3.1 Rules & thresholds

In plain English. Every number, switch, list and piece of text the QA system uses, apart from formulas, which have their own tab. That is about 100 rules, grouped by subject. The **Group** filter uses the catalogue names (*QA · Fees and payments*, *QA · Institutions and documents*, *QA · Workflow and SLA*, *QA · Decisions and letters*, *QA · Capacity engine*, *QA · Complaints*, *QA · Eligibility and qualifications*, *QA · Credentials, portal and impersonation*, *QA · Interaction*, *QA · Settings*, *QA · Requests and SSQ*, *Backups*); the list itself shows friendlier headings for some of them, such as *Money in*, *Deadlines and routing*, *Who may serve*, *Portal and sign-in* and *Screens*. Each rule's title is made from its key (for example `qa.query.default_due_days` is shown as *Default due days*).

At the top, with no filter applied, a card called **Institution self-registration** holds the three portal registration settings (section 6, walkthrough W13).



Rules and thresholds: the Institution self-registration card at the top, with the switch, the contact e-mail and phone, and the state line

Each rule shows a title, a description, its key, its current value, its badge and, if a change is waiting, a link *PA-… awaiting confirmation*. Use the search box (key or description), the **Group**, **Type** and **Control** filters and **Only what I can edit** to find a rule. Then:

- **On/off rules (direct):** flip the switch; it saves at once.
- **Other direct rules:** press **Edit**, enter the new value, **Save**.
- **Dual-control rules:** press **Edit** (or **Change** for a switch), enter the new value and a **reason for the second officer** (for example the memo reference), then **Send for confirmation**.
- **History** shows past values with who changed them, and recent pending items.

Who can change it. Holders of the rule's edit permission: `qa_core.edit_qa_settings` for almost all rules. Exceptions: `qa.complaints.fallback_department` needs `complaints.manage_routing_rules`; `qa.payment.statement_column_map` needs `payments.import_bank_statement`; the four `qa.backup.*` rules need `backups.manage_destinations` (they are normally edited on Settings → Backups); `qa.payment.provider_confirmation_is_evidence_only` is fixed.

How changes take effect. Direct rules apply from the next read; dual-control rules apply when confirmed. Each value is checked against its type (whole number, decimal, yes/no, JSON list or object, date, text). The system does **not** check whether a number is sensible, so a reminder percentage of 150 or a department code that does not exist is accepted. Check the meaning yourself and test on a training copy.

What it touches. Almost everything: deadlines and reminders (D5 of the master guide), payments (D3), portal access, eligibility of resource persons, the capacity engine (D10), letter wording and the complaint clock. Section 8 lists the most important keys.

Technically. Views `qa:settings_rules`, `qa:setting_edit`, `qa:setting_save`, `qa:setting_history`; rows from `settings_service.qa_entries()` (excluding `value_type="formula"`). The featured keys are `FEATURED_KEYS` in `qa_core/views.py`. The catalogue of keys, defaults and controls is `qa_core/settings_catalogue.py`, seeded by `seed_startup` with get-or-create (an administrator's edits are never overwritten).

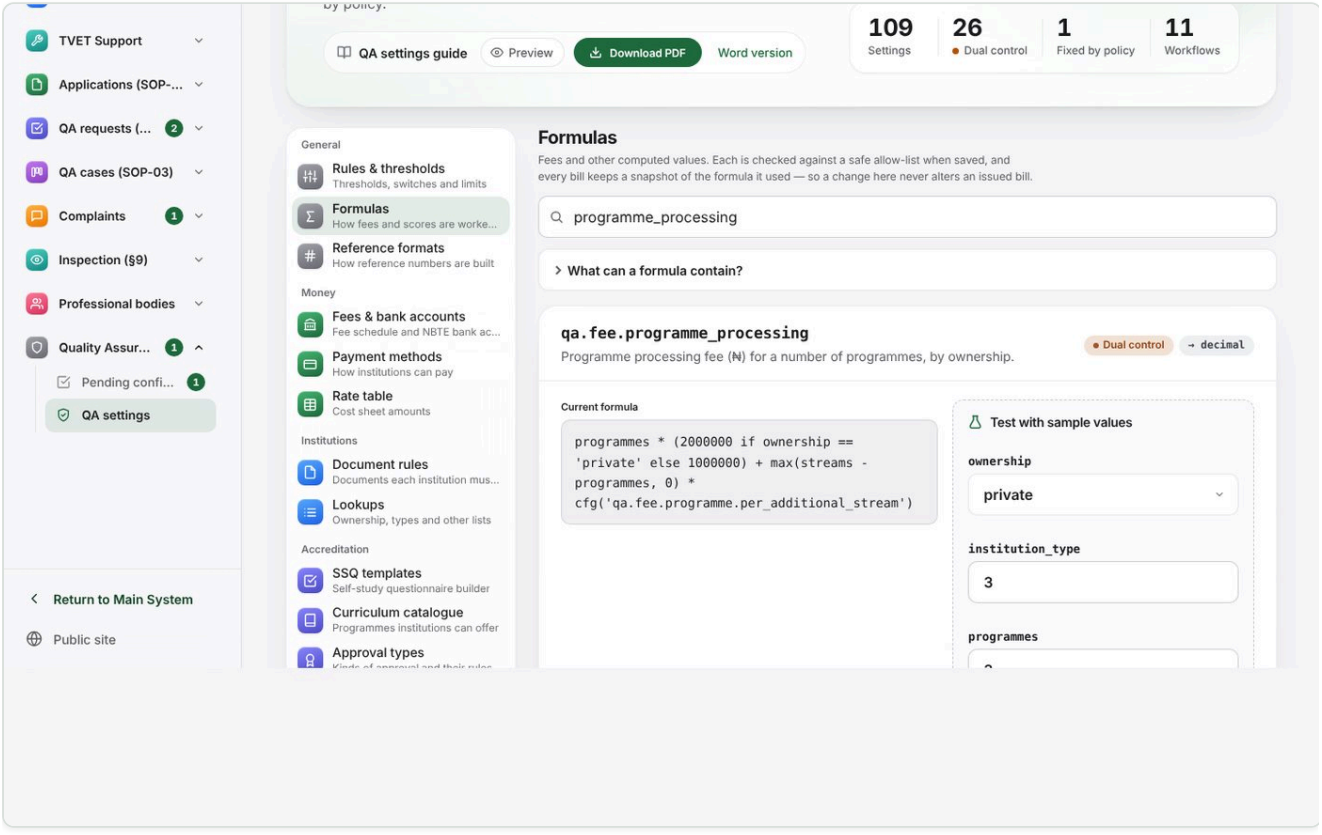
3.2 Formulas

In plain English. The calculations the system uses, each written as one short formula. The seeded formulas are:

Formula	What it works out	Variables	Control
`qa.fee.institutional_processing`	SOP-01 institutional processing fee (₦)	the six fee variables	Dual
`qa.fee.programme_processing`	Programme processing fee (₦)	the six fee variables	Dual
`qa.fee.qa_request_processing`	SOP-02 QA request processing fee (₦)	the six fee variables	Dual

Formula	What it works out	Variables	Control
`qa.fee.<code>`	Each fee added with **Add a fee**	those ticked when it was added	Dual
`qa.capacity.additional_streams`	Additional streams rooms can carry	`standard_hours`, `total_size_factor`, `contact_hours`, `approved_streams`, `other_load`	Dual
`qa.capacity.rule_of_thumb`	Indicative minimum number of lecturers	`streams`	Direct
`qa.ratio.staff_student`	Students per counted lecturer	`students`, `counted_staff`	Direct
`qa.matching.rank_score`	Ranking of available people in the finder (lower first)	`visits_this_year`, `days_since_last`, `rating`	Direct

Each formula card shows the **current formula**, its **variables**, its badge and its result type, a box to **propose a new formula**, and a **Test with sample values** panel. **Run test** works out the formula in the editor (not the saved one) with the sample values you type, shows the result and lists every other setting the formula read with `cfg('…')` and its value. Nothing is saved by a test.



Formulas: the programme processing fee with its current formula, variables and the Test with sample values panel showing a result and the settings it read

The formula language. Formulas use numbers, text in quotes, the declared variables and:

You can use	Example
`+ - * /`, `//` (whole division), `%` (remainder), `**` (power, whole exponent up to 4)	`programmes * 1000000`
Comparisons `== != < <= > >=`	`ownership == 'private'`
`and`, `or`, `not`	`programmes > 2 and ownership == 'state'`
`a if condition else b`	`4000000 if ownership == 'private' else 2000000`
`min(…)`, `max(…)`, `round(x)` or `round(x, places)`, `ceil(x)`, `floor(x)`, `abs(x)`, `clamp(x, low, high)`	`max(streams - programmes, 0)`
`cfg('key')` to read another setting (not in capacity formulas)	`cfg('qa.fee.programme.per_additional_stream')`

Anything else (attribute access such as `x.y`, square brackets, imports, keyword arguments) is refused with a plain message such as *Unknown name 'stream'. Declared variables: ….* Text is compared exactly, so `'Private'` is not the same as `'private'`.

Who can change it. Propose: `qa_core.edit_qa_settings`. Confirm (dual-control formulas): an officer of the same department with `qa_core.edit_qa_dual_control_settings`, rank 50 or more. Anyone who can open QA settings can test a formula.

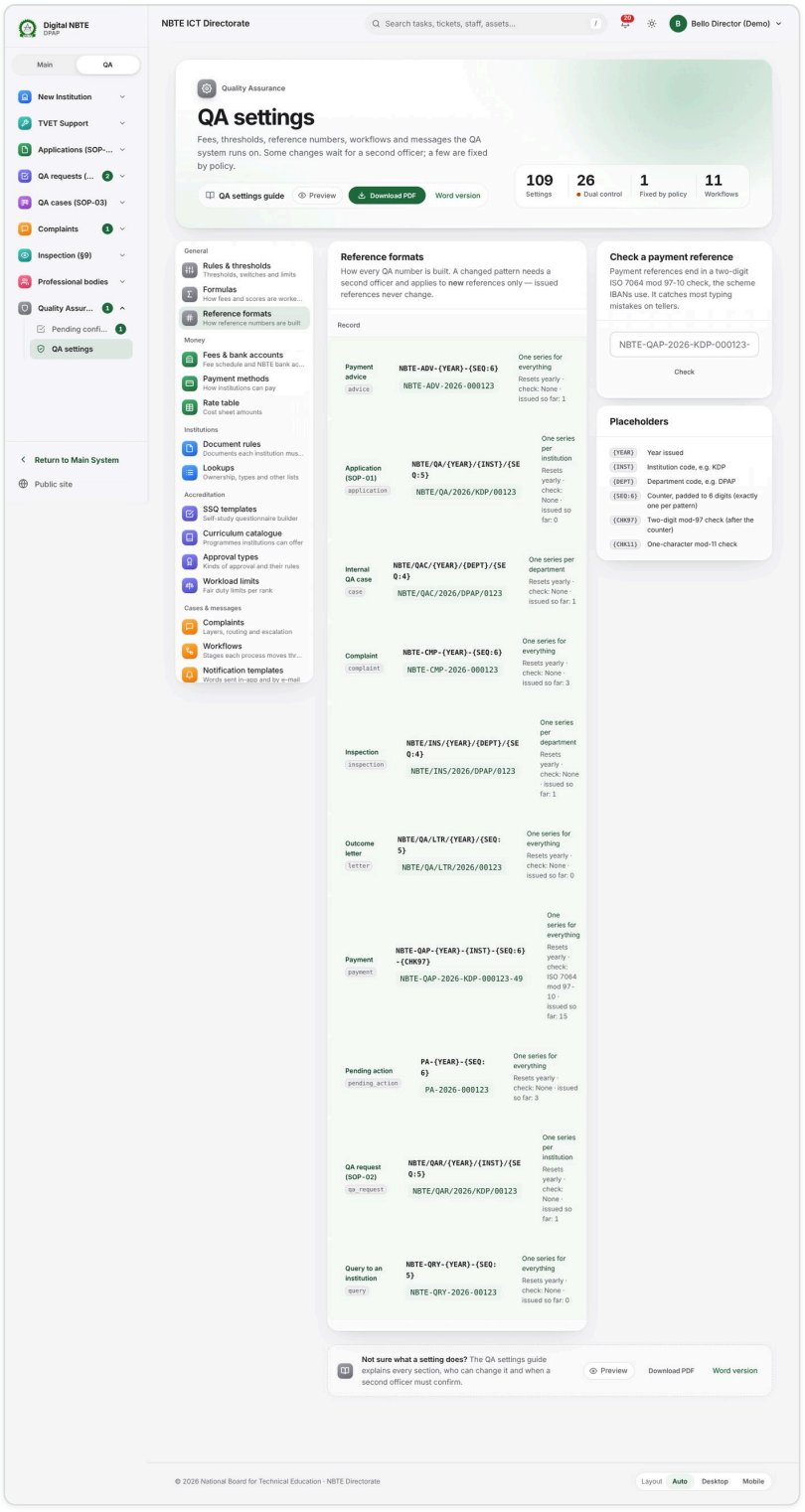
How changes take effect. When confirmed (or saved, for direct ones). The variables a formula may use are fixed when the formula is created; editing a formula changes only its text.

What it touches. Fee formulas set the amount of every bill issued afterwards and the portal's live estimate; the capacity formulas feed every capacity assessment run afterwards; the rank score orders the availability finder.

Technically. `qa_core/formula.py` parses with Python's `ast` module and evaluates an allow-list of node types with `Decimal` arithmetic (no `eval()`); limits: 1,000 nodes, 50 ms, 4,000 characters, `cfg()` nesting 10 with cycle detection. A formula setting's value is JSON `{"expr", "vars", "returns"}`. Views `qa:settings_formulas`, `qa:formula_test`, `qa:formula_save`. Capacity formulas are evaluated with a getter that refuses `cfg()` (`accreditation/capacity.py`).

3.3 Reference formats

In plain English. How every QA reference number is built. Each format has a **pattern** made of fixed text and placeholders, a **reset** period (yearly or never), an optional **check digit**, the **allowed characters** and a **numbering scope** (one series for everything, one per institution, or one per department). The page shows a sample of each format, the last number issued, the list of placeholders and a box to **check a payment reference**.



Reference formats: each format with its pattern, a sample reference, reset period, check digit, numbering scope and last number issued, the placeholder list and the payment-reference checker

Placeholder	Becomes
`{YEAR}`	The four-digit year
`{INST}`	The institution code, e.g. `KDP`
`{DEPT}`	The department code in capitals, e.g. `DPAP`

Placeholder	Becomes
`{SEQ:n}`	The running number padded to *n* digits (`{SEQ}` = no padding). **Exactly one is required**
`{CHK97}`	Two-digit ISO 7064 mod 97-10 check (the scheme IBANs use)
`{CHK11}`	One-character mod-11 check
`{CHK}`	The check named by the format's check-digit method

The installed formats are:

Code	Pattern	Numbered
`application`	`NBTE/QA/{YEAR}/{INST}/{SEQ:5}`	Per institution, yearly
`qa_request`	`NBTE/QAR/{YEAR}/{INST}/{SEQ:5}`	Per institution, yearly
`payment`	`NBTE-QAP-{YEAR}-{INST}-{SEQ:6}-{CHK97}`	One series, yearly, mod 97
`case`	`NBTE/QAC/{YEAR}/{DEPT}/{SEQ:4}`	Per department, yearly
`inspection`	`NBTE/INS/{YEAR}/{DEPT}/{SEQ:4}`	Per department, yearly
`complaint`	`NBTE-CMP-{YEAR}-{SEQ:6}`	One series, yearly
`letter`	`NBTE/QA/LTR/{YEAR}/{SEQ:5}`	One series, yearly
`advice`	`NBTE-ADV-{YEAR}-{SEQ:6}`	One series, yearly
`pending_action`	`PA-{YEAR}-{SEQ:6}`	One series, yearly
`query`	`NBTE-QRY-{YEAR}-{SEQ:5}`	One series, yearly

Who can change it. Propose: `qa_core.manage_reference_formats`. Confirm: same department, `qa_core.edit_qa_dual_control_settings`, rank 50 or more. Every change is dual control.

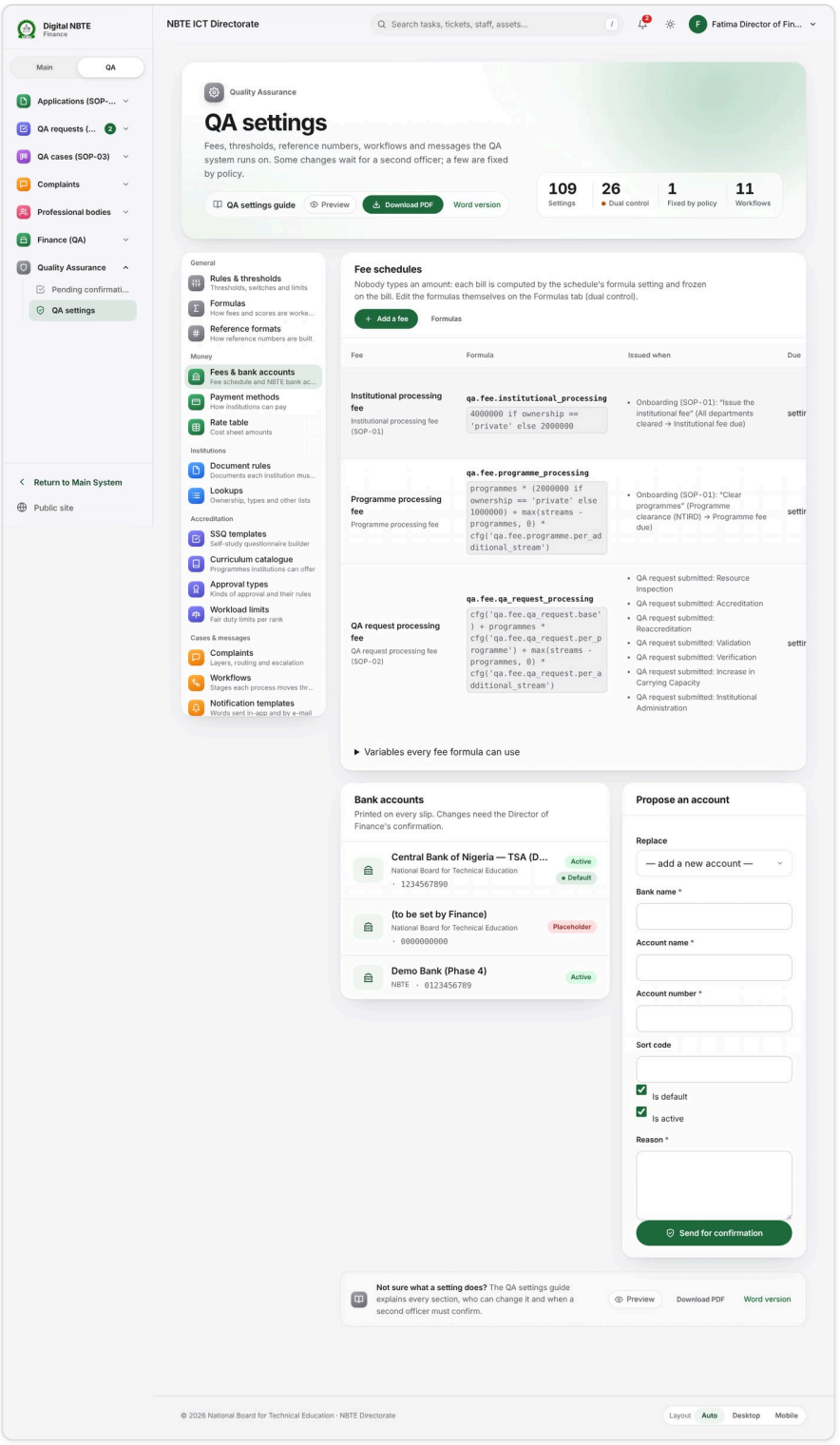
How changes take effect. When confirmed, for references issued from then on. Issued references never change. The running number carries on from where it was, because the counter belongs to the format, the year and the scope, not to the pattern.

What it touches. Every new application, request, bill, case, inspection, complaint, letter, payment advice, pending item and query. Payment references are also given to Remita and Paystack and typed by institutions on bank tellers, which is why they are restricted to `A-Z0-9.=-` and carry a check digit.

Technically. `qa_core.ReferenceFormat` and `ReferenceCounter`; `qa_core/references.py` (`validate_pattern`, `render`, `issue`, `verify`). Views `qa:settings_references`, `qa:reference_edit`, `qa:reference_check`. Policy `qa_core.update_reference_format`. Counters are incremented atomically so two officers never receive the same number.

3.4 Fees & bank accounts

In plain English. The **fee schedules** and the **NBTE bank accounts** printed on payment slips. Nobody ever types the amount of a bill: each schedule points at a formula setting, and the formula computes the amount when the bill is issued.



Fees and bank accounts: fee schedules with their formula, when each is issued, due days, Remita service type and account, the Add a fee button, the fee variables, and the NBTE bank accounts

Each schedule row shows the fee's name and slip text, its formula key and expression, **Issued when** (the workflow step that issues it, *QA request submitted: …*, or *Not issued automatically — Finance issues it from an institution's bills page (Issue a bill)*), due days, Remita service type and bank account. A schedule marked *Unconfirmed — bills blocked* cannot be billed until its confirmation switch is on (the QA request fee waits for `qa.fee.qa_request_processing.confirmed`).

The three seeded schedules:

Code	Name	Formula setting	Blocked until
`institutional_processing`	Institutional processing fee	`qa.fee.institutional_processing`	—
`programme_processing`	Programme processing fee	`qa.fee.programme_processing`	—
`qa_request_processing`	QA request processing fee	`qa.fee.qa_request_processing`	`qa.fee.qa_request_processing.confirmed` is true

Variables every fee formula can use (open the panel under the table):

Variable	Meaning
`ownership`	Ownership code of the institution: `'federal'`, `'state'` or `'private'` (`'public'` if none is recorded)
`institution_type`	Institution type code, e.g. `'polytechnic'`, `'monotechnic'`
`programmes`	Number of programmes the bill is for
`new_programmes`	How many of those programmes are new (proposed)
`streams`	Total streams asked for across those programmes (each programme counts at least 1)
`max_streams`	The most streams asked for in any one of those programmes

Three things can be done here:

- **Change…** on a schedule: change which formula setting it uses, its due days, Remita service type, bank account, slip text or whether it is active. This is dual control (*Change Fee Schedule*). To change the *amount*, change the formula on the **Formulas** tab instead.
- **Add a fee:** a new fee with its own formula (walkthrough W3).
- **Propose an account:** add or replace an NBTE bank account (bank, account name, 10-digit NUBAN, sort code, default, active, reason). Dual control (*Change Bank Account*). Until a real account is confirmed, slips cannot be issued; bills wait at *issued* and are published automatically once it is.

Who can change it. Propose (all three): `payments.view_obligation` (Finance). Confirm: the Director of Finance (`payments.manage_fee_schedule` for fees, `payments.manage_bank_accounts` for accounts; rank 100; same department).

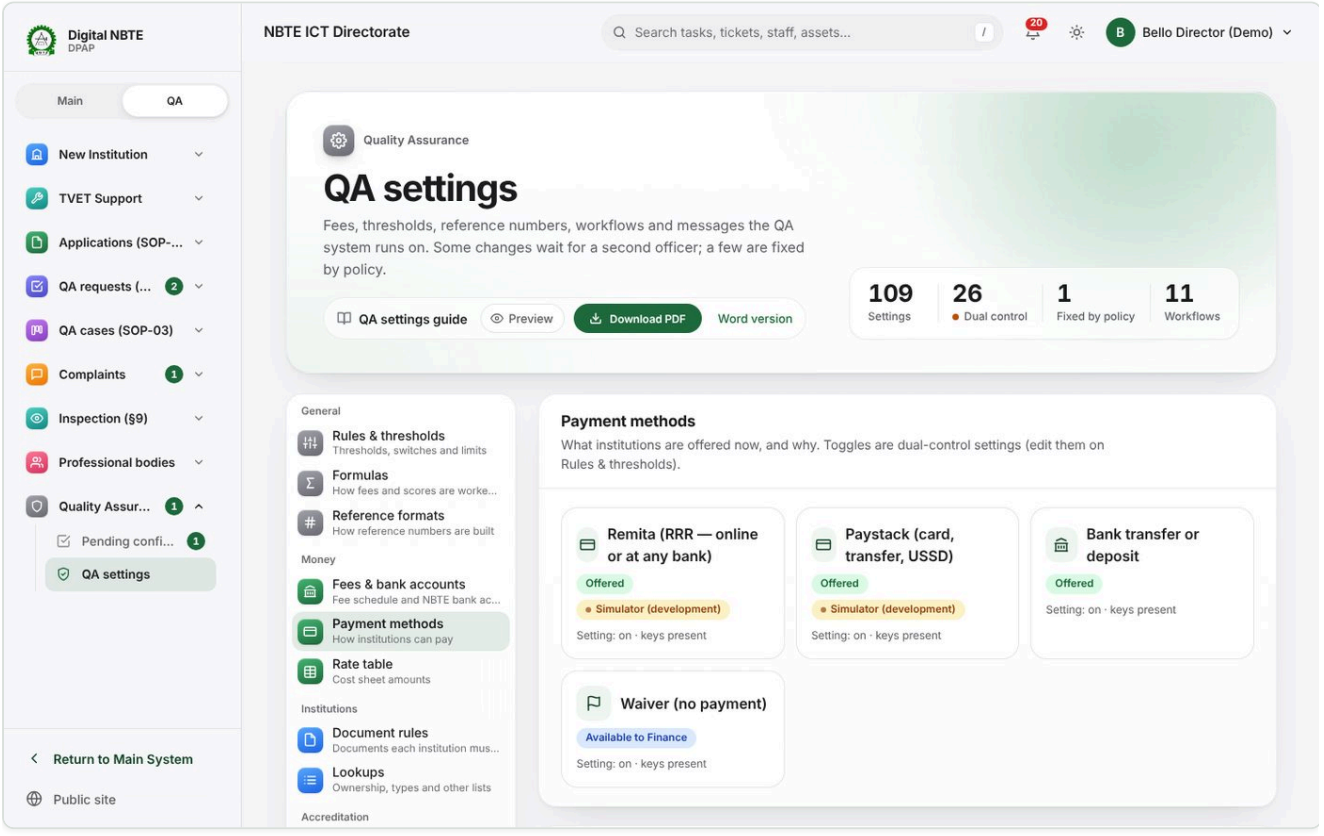
How changes take effect. On confirmation. A new or changed schedule is used by the next bill; bills already issued keep their amount, slip text and account.

What it touches. Every bill (SOP-01 institutional and programme fees, SOP-02 QA request fees, bills issued by hand), the payment slip and advice, the portal's live estimate and the bills Finance confirms.

Technically. `payments.FeeSchedule` (`code`, `formula` = setting key, `confirmed_setting`, `due_days`, `remita_service_type_id`, `revenue_account`, `is_active`), `payments.BankAccount`. Views `qa_set:fees`, `qa_set:fee_form`, `qa_set:bank_form`, `qa_set:fee_new`, `qa_set:fee_test`. Bills are created by `payments.services.issue_obligation()`, which evaluates `evaluate_setting(schedule.formula, fee_variables(institution, …))` and stores the snapshot. Which events issue a fee: `payments.fee_admin.issuing_events()` (workflow effects `create_payment_obligation:<code>` and each QA type's `fee_schedule`).

3.5 Payment methods

In plain English. Which ways of paying institutions are offered now, and why. Each method (Remita, Paystack, bank transfer, offline/waivers) shows *Offered* or *Not offered*, whether its setting is on and whether its keys are present on the server. A red notice appears when a method is switched on but its server keys are missing; it is not offered until ICT sets them and restarts. If every other method is off, **bank transfer is offered anyway** (an enforced fallback). Below, **Payment rules** lists the payment settings with their badges.



Payment methods: which methods are on, and the payment rules with their control badges

Who can change it. This page is read-only. The switches and rules are edited on **Rules & thresholds** (the **Edit on Rules & thresholds** link filters to `qa.payment`). Most are dual control.

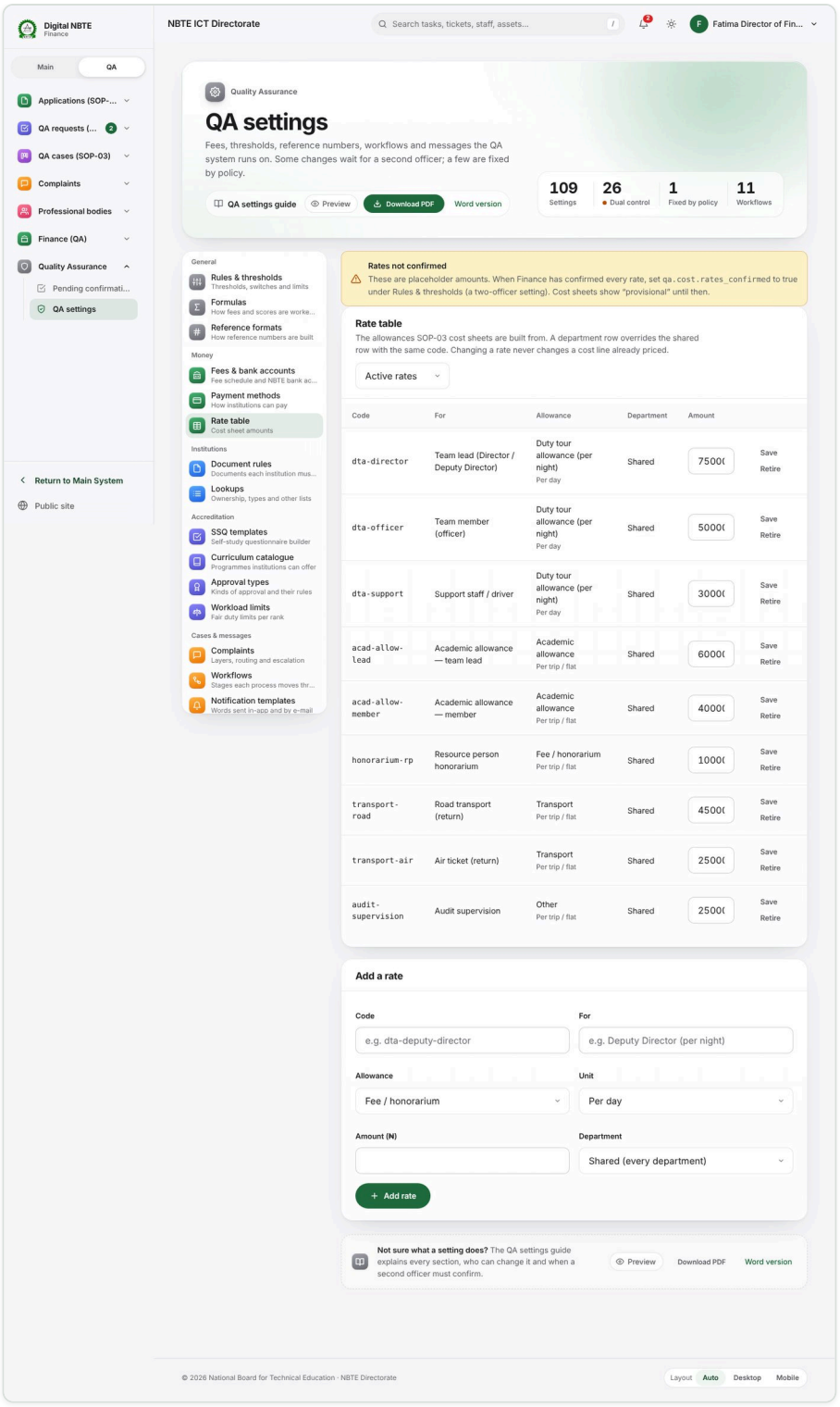
How changes take effect. When confirmed, for the next time an institution opens a bill. Payments already in progress are not affected.

What it touches. The payment page on the portal, the order in which methods are listed (`qa.payment.method_order`), who bears the Paystack fee, partial payments, the 24-hour Finance confirmation window, the second check above ₦5,000,000, the receipt requirement and waivers.

Technically. View `qa_set:methods` (`payments/views.py`); provider status from `payments.providers.status_report()` and `enabled_methods()`. Keys are listed in section 8.

3.6 Rate table

In plain English. The allowance rates SOP-03 visit cost sheets are built from: duty tour allowance per day, academic allowances, honoraria, transport and other amounts. Each rate has a **code**, **who it is for**, a **kind**, a **unit** (per day, or per trip/flat), an **amount** and an optional **department**. A department row overrides the shared row with the same code for that department's cases. While `qa.cost.rates_confirmed` is false, a notice says the amounts are placeholders and cost sheets show *provisional*.



Rate table: the placeholder-amounts notice, each allowance with who it is for, kind, unit, amount and department, Save and Retire, and Add a rate

The seeded rates (placeholders until Finance confirms them):

Code	For	Kind	Unit	Amount (₦)
`dta-director`	Team lead (Director / Deputy Director)	Duty tour allowance	Per day	75,000

Code	For	Kind	Unit	Amount (₦)
`dta-officer`	Team member (officer)	Duty tour allowance	Per day	50,000
`dta-support`	Support staff / driver	Duty tour allowance	Per day	30,000
`acad-allow-lead`	Academic allowance — team lead	Academic allowance	Per trip	60,000
`acad-allow-member`	Academic allowance — member	Academic allowance	Per trip	40,000
`honorarium-rp`	Resource person honorarium	Fee / honorarium	Per trip	100,000
`transport-road`	Road transport (return)	Transport	Per trip	45,000
`transport-air`	Air ticket (return)	Transport	Per trip	250,000
`audit-supervision`	Audit supervision	Other	Per trip	25,000

Who can change it. `accreditation.manage_rate_table` (Director of Finance (QA) role, QA Settings Admin group). Rates save directly. Confirming that the whole table is final (`qa.cost.rates_confirmed`) is a dual-control rule.

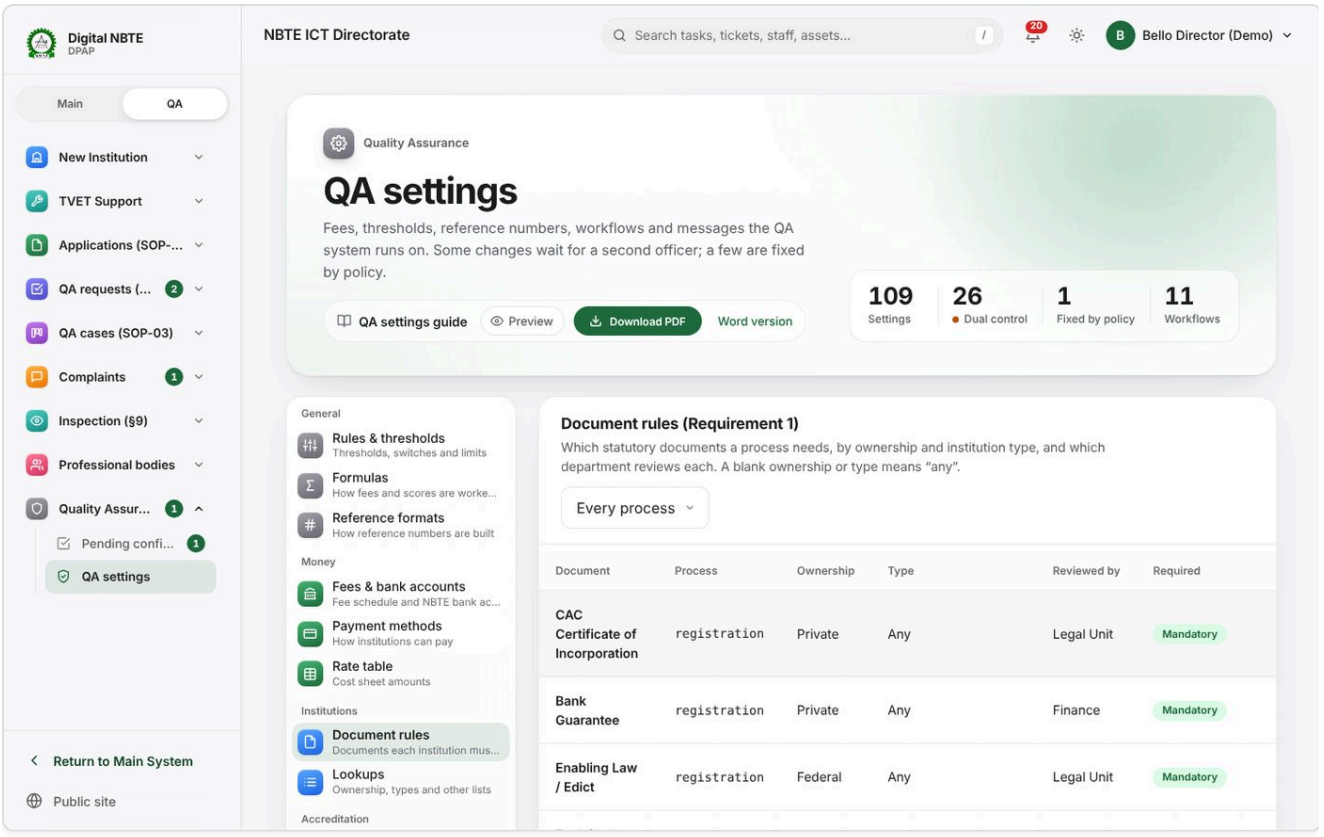
How changes take effect. At once, for cost lines added afterwards. *Cost lines already priced keep their amounts.* **Retire** hides a rate from new cost sheets; **Restore** brings it back.

What it touches. Stage 2 (generate the cost) and stages 5 and 6 (audit) of SOP-03, and therefore the total the ES approves at stage 7.

Technically. `accreditation.RateTable` (unique on `code` + `department`); views `qa_set:rates`, `qa_set:rate_edit`. `accreditation.case_services.rates_for(case)` returns active shared rates overridden by the owning department's rows; `line_amount()` = amount × days for per-day rates.

3.7 Document rules

In plain English. Which statutory documents a process needs, by ownership and institution type, and which department reviews each. A blank ownership or type means *any*. Each rule says the **process** (for example `registration`, `ownership_change`), the **document kind**, whether it is **mandatory**, the **reviewing department**, **ask again after (months)** (blank = never ask twice), notes and whether it is active. When two rules name the same document for an institution, the mandatory one wins.



Document rules: each required document by process, ownership and type, with the reviewing department and whether it is mandatory

Who can change it. `institutions.manage_document_requirements` (QA Settings Admin role). Rules save directly (**Save rule**).

How changes take effect. At once. The list of required documents is worked out live every time it is needed, so an institution's *Documents* page and the checks before an application can be submitted change immediately, for existing institutions too.

What it touches. SOP-01 onboarding (what an institution must upload before submitting), departmental clearance (who reviews each document), ownership changes (newly required documents become an outstanding request and, with `qa.ownership_change.hold_open_cases` on, the open application is held) and the downloadable institution guides.

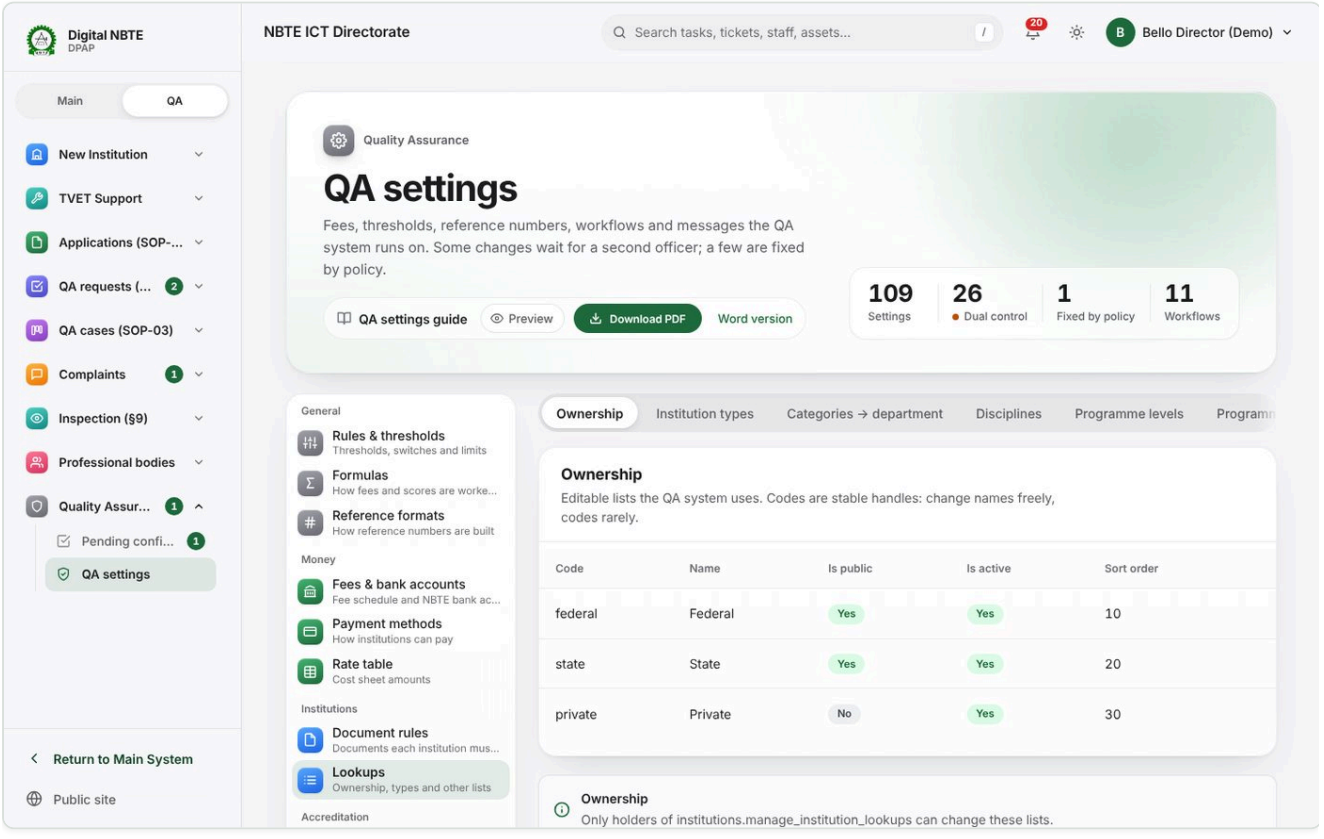
Technically. `institutions.DocumentRequirement`; view `qa_set:documents`; `institutions.services.requirements_for(inst, process)`.

3.8 Lookups

In plain English. The lists used in forms across the system. Choose the list at the top:

List	Fields	Used for
Ownership	code, name, public?, active, order	Registration, fees (`ownership` variable), document rules

List	Fields	Used for
Institution types	code, name, default department, active, order	Registration, the handling department, fees (`institution_type`)
Categories → department	code, name, institution type, routing department, active, order	Which department handles an institution
Disciplines	code, name, active, order	Programmes, matching of resource persons
Programme levels	code, name, active, order	Programmes, letters
Programme categories	name, discipline, students per stream, technology based?, active	Programmes, the capacity engine (students per stream), batch decisions
Document kinds	code, name, description, accepted file types (pdf, jpg, jpeg, png, docx, xlsx), largest size (MB), active, order	Uploads, document rules, SSQ prefill



Lookups: ownership values with code, name, shown on the portal and in action, and sort order

Who can change it. `institutions.manage_institution_lookups` (QA Settings Admin role). Changes save directly.

How changes take effect. At once. *Codes are stable handles: change names freely, codes rarely.* A code is what formulas, document rules, routing conditions and imports refer to (for example the fee

formulas compare `ownership == 'private'`), so renaming a code silently breaks them. To retire a value, untick **active** instead of deleting or renaming it.

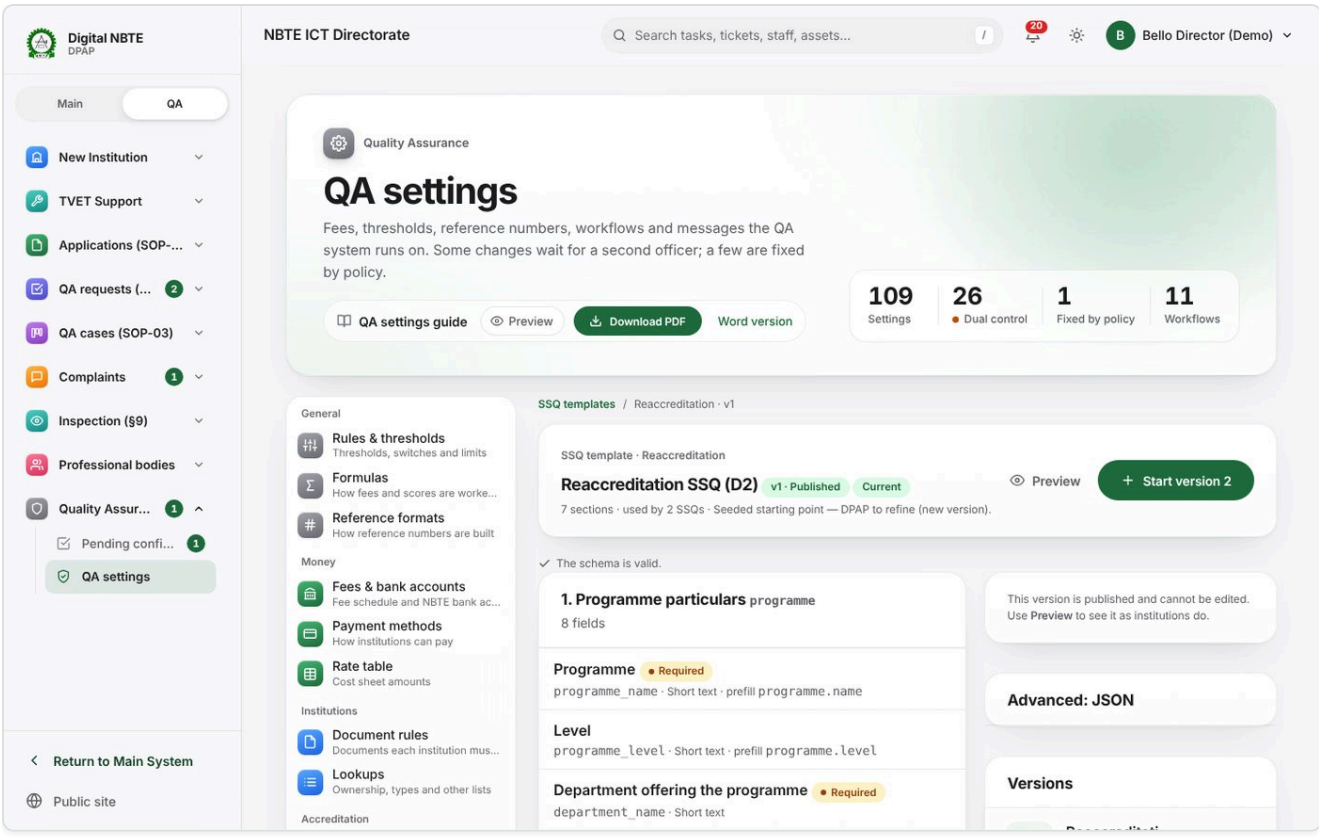
Technically. `institutions.forms.LOOKUP_FORMS` maps the list name (`ownership`, `type`, `category`, `discipline`, `level`, `programme-category`, `document-kind`) to its model; view `qa_set:lookups` / `qa_set:lookups_kind` at `/qa/settings/lookups/<list>/`.

3.9 SSQ templates

In plain English. The self-study questionnaire (SSQ) for each QA type: Resource Inspection, Accreditation, Reaccreditation, Validation, Verification, Increase in Carrying Capacity and Institutional Administration. *A published version is never edited: start a new version, change it, publish it. SSQs already open keep the version they started with.*

The list shows each QA type with its current version and any draft. **New version** (or **Create template** for a type with none) copies the current version into a draft. The **builder** then lets you:

- **Add section**, edit its title and description, choose whether it may be sent to an academic department, move it up or down, or remove it;
- **Add field** of a chosen type: short text, long text, e-mail, phone, web address, number, whole number, date, one choice, several choices, yes/no, file (evidence, tied to a document kind), catalogue quantities, or table (with columns); set the label, help, required, validation limits and a **prefill** source (institution details, contacts, programme details or an accepted document);
- **Preview** the questionnaire as the institution will see it;
- **Advanced: JSON** to paste a whole schema;
- **Publish** (only when the schema is valid) or **Discard draft**.



The SSQ template builder: sections and questions of a questionnaire, with question types and required flags

Who can change it. Start and edit drafts: `accreditation.manage_ssq_templates` (Directors, Curriculum Officers, QA Settings Admin group). Publish: `accreditation.publish_ssq_template` (Directors). Publishing is not a pending confirmation, but because administrators and curriculum officers cannot publish, a Director normally reviews the draft before publishing it.

How changes take effect. Drafts affect nobody. On **Publish** the draft becomes the current version for its QA type and the previous version is retired (kept, never changed). SSQs are opened, one per programme, when a QA request's fee is confirmed; they use the version current at that moment. Only one draft per QA type can exist at a time, and a draft that is in use cannot be discarded.

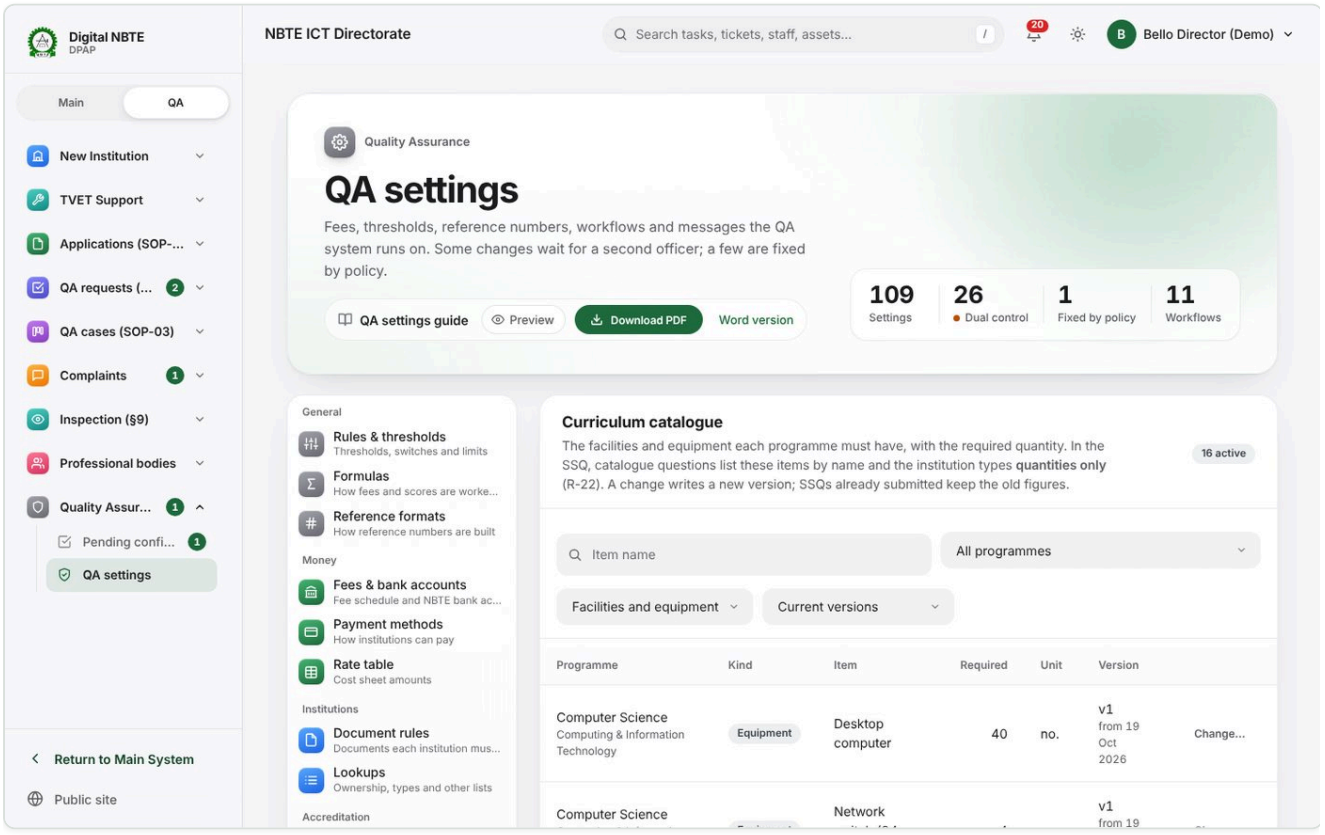
What it touches. What institutions fill in for SOP-02, the review queue, prefill from the institution's record, catalogue questions and the capacity engine's starting values (classrooms, facilities, library, staff lists).

Technically. `accreditation.SSQTemplate` (`qa_type`, `version`, `status` draft/published/retired, `is_current`, `schema`); `accreditation/builder.py` (`new_draft`, `save_draft`, `publish`, `discard_draft`); schema validation `accreditation.ssq.validate_schema`; views `qa_set:ssq_templates`, `qa_set:ssq_template`, `…_section`, `…_field`, `…_json`, `…_publish`, `…_discard`, `…_preview`. Publishing sets `QAType.ssq_template`; `accreditation.qa_services.on_open_ssqs` uses it.

3.10 Curriculum catalogue

In plain English. The facilities and equipment each programme must have, with the required quantity. Items belong either to a **programme** by name (for example *Computer Science*) or to a **programme category** (applies to every programme in the category that has no list of its own). In the

SSQ, a catalogue question lists these items by name and the institution types only the quantity it has; the reviewer sees at once where it falls short.



The curriculum catalogue: equipment and facilities per programme with the quantity required

Add an item: category or programme name, facility or equipment, name, required quantity, unit. **Change…** on an item: a new quantity, unit or name is saved as the next **version** and the old one is retired (*Save as version n*). **Retire item** removes it from new SSQs.

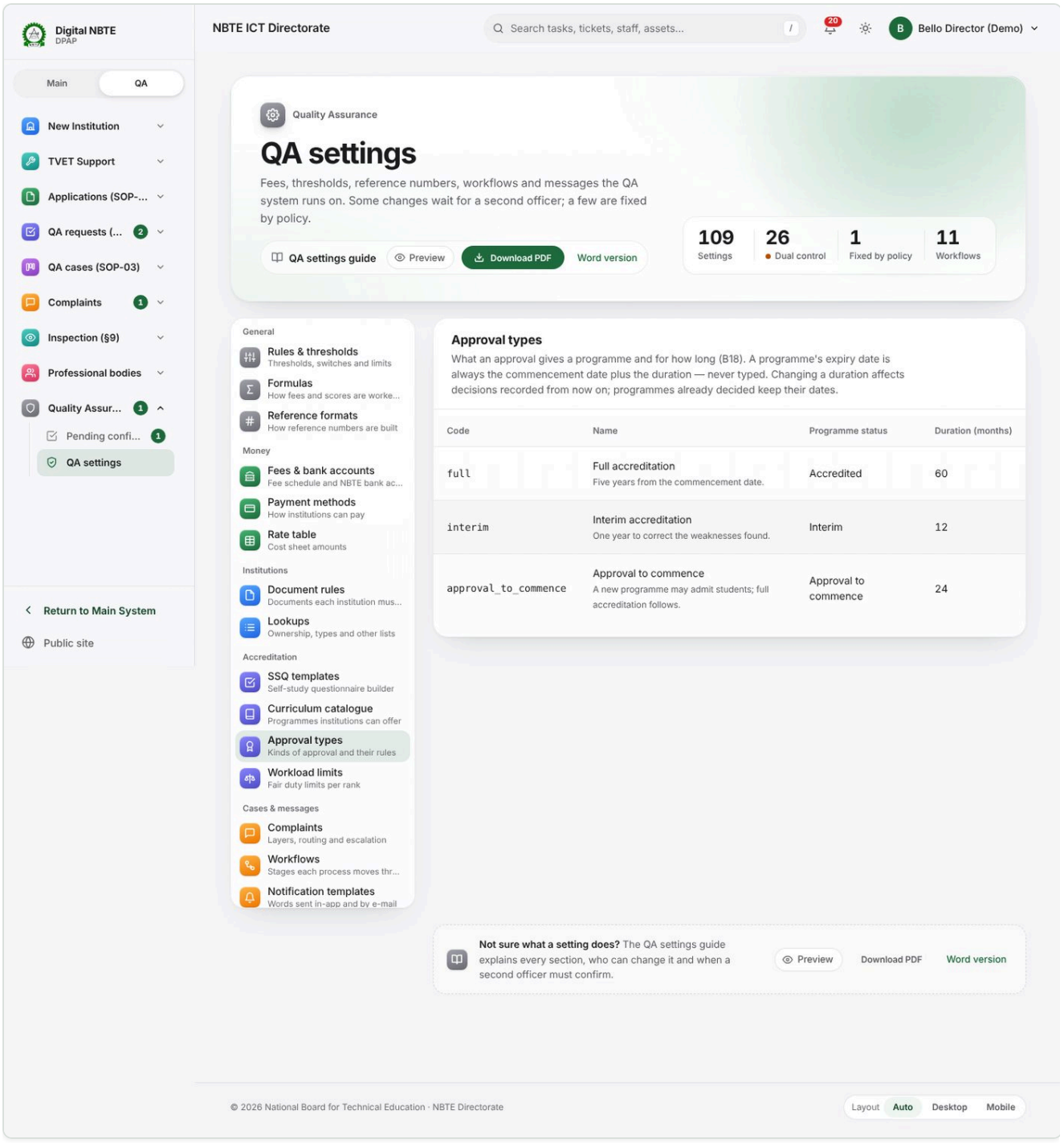
Who can change it. `accreditation.manage_catalogue` (Directors, Curriculum Officers, QA Settings Admin group). Changes save directly.

How changes take effect. At once for SSQs being filled and opened afterwards. *SSQs already submitted keep the old figures* (their snapshot stores the item name, required quantity and version). Because a revised line is a new item, an institution in the middle of filling an SSQ may need to re-enter the quantity for that line.

Technically. `accreditation.CatalogueItem` (`category`, `programme_name`, `kind`, `name`, `required_quantity`, `unit`, `version`, `effective_from`, `is_active`); `builder.create_item`, `revise_item`, `retire_item`; `accreditation.ssq.catalogue_items(field, programme)` picks the programme's own list, else the category's. Views `qa_set:catalogue`, `qa_set:catalogue_item`.

3.11 Approval types

In plain English. What an approval gives a programme and for how long. *A programme's expiry date is always the commencement date plus the duration — never typed.*



Approval types: Full accreditation 60 months, Interim accreditation 12 months and Approval to commence 24 months, each with the programme status it gives, and Add an approval type

Code	Name	Duration	Programme status it gives
`full`	Full accreditation	60 months	Accredited
`interim`	Interim accreditation	12 months	Interim
`approval_to_commence`	Approval to commence	24 months	Approval to commence

You can change a type's name, duration (1 to 240 months) and programme status (**Save**), switch it off or on, or **Add an approval type** (code, name, duration, status, description).

Who can change it. `accreditation.manage_approval_types` (QA Settings Admin role and group). Changes save directly.

How changes take effect. At once. The expiry date is worked out when a decision is recorded and **again when it is applied to the programme at stage 17**, using the duration at that moment. So a new duration applies to every programme whose decision is applied after you save, including decisions already recorded by the CQAC or the ES but not yet applied. Programmes already updated at stage 17 keep their expiry dates. (The screen's wording, *affects decisions recorded from now on*, is a slight simplification.)

What it touches. Stage 14–16 decisions, the programme's status, expiry date and re-accreditation window (expiry minus `qa.programme.reassessment_window_months`), outcome letters (`{approval_type}`, `{expiry}`), renewal reminders and the daily expiry job.

Technically. `accreditation.ApprovalType.expiry_for(commencement)` = `add_months(commencement, duration_months)`; `Decision.save()` derives `expiry_date`; `accreditation.decisions._apply_one()` sets `programme.expiry_date` and `eligible_for_reassessment_at`. Views `qa_set:approval_types`, `qa_set:approval_type_edit`.

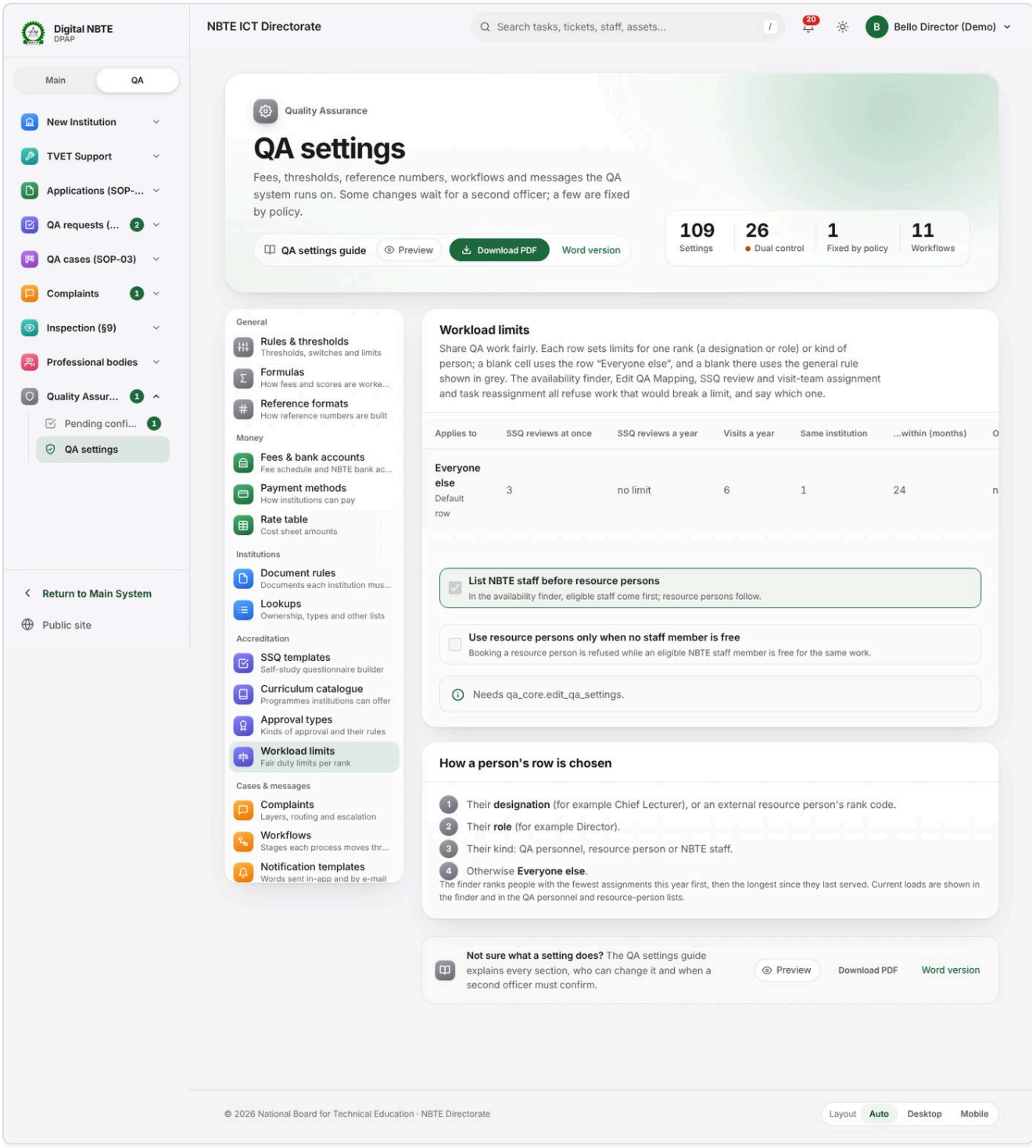
3.12 Workload limits

In plain English. *Share QA work fairly.* One row per rank (a designation or role) or kind of person, with six columns:

Column	Meaning	If blank everywhere, falls back to
SSQ reviews at once	Open SSQ reviews a person may hold at the same time	`qa.resource_person.max_open_ssq_per_officer` (3)
SSQ reviews a year	SSQ reviews in one calendar year	No limit
Visits a year	QA visits in one calendar year	`qa.resource_person.max_visits_per_year` (6)
Same institution	Times the same person may serve the same institution within the period	1
…within (months)	The period for the same-institution limit	`qa.matching.same_institution_cooldown_months` (24)
Open tasks	Open tasks of any kind before new QA work is refused	No limit

A blank cell uses the **Everyone else** row; a blank there uses the general rule shown in grey. A person's row is the first that matches, in this order: their **designation**, their **role**, their **kind** (*QA personnel*, *Resource persons*, *NBTE staff*), then *Everyone else*. Two switches sit under the table:

List NBTE staff before resource persons and **Use resource persons only when no staff member is free**.



Workload limits: the table of limits per rank with the Everyone else row, the general rules shown in grey, the two staff-first switches and the reason

Who can change it. `qa_core.edit_qa_settings`. **Add a rank** and **Remove** only change the editor; nothing is stored until **Save limits**. The table and switches are direct settings (they become dual control only if an administrator marks them so).

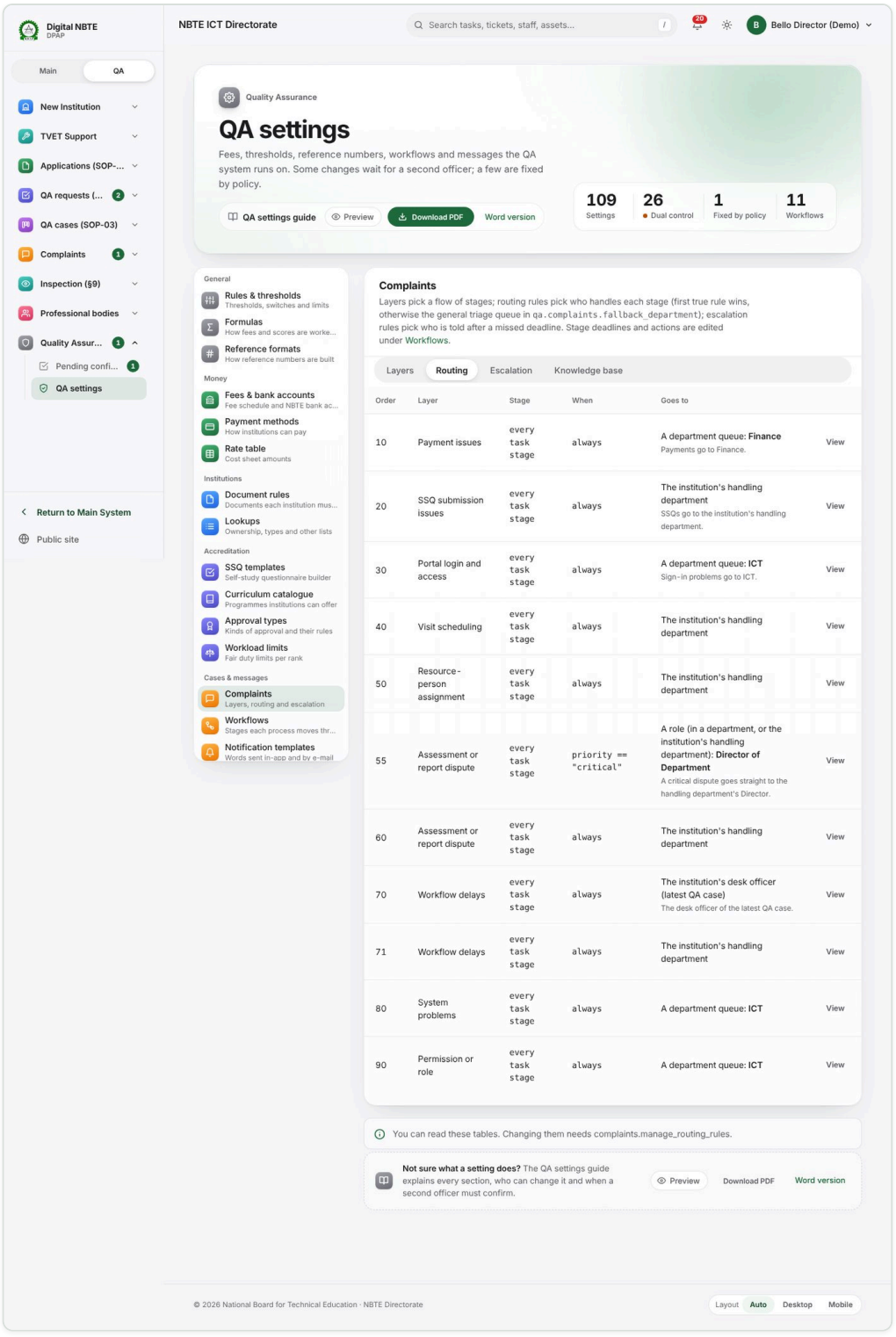
How changes take effect. From the next assignment. *The availability finder, Edit QA Mapping, SSQ review and visit-team assignment and task reassignment all refuse work that would break a limit, and say which one.* There is no override.

Technically. One JSON setting `qa.workload.limits` (`{"rows": [{"key": "default"}, {"key": "director", "max_visits_per_year": 4}]}`) plus `qa.workload.staff_first` and `qa.workload.resource_persons_only_when_no_staff`; `inspection/workload.py` (`FIELDS`, `table`, `row_for`, `limit`, `problems_for`); view `qa_set:workload`, saved through `settings_service.save_setting`.

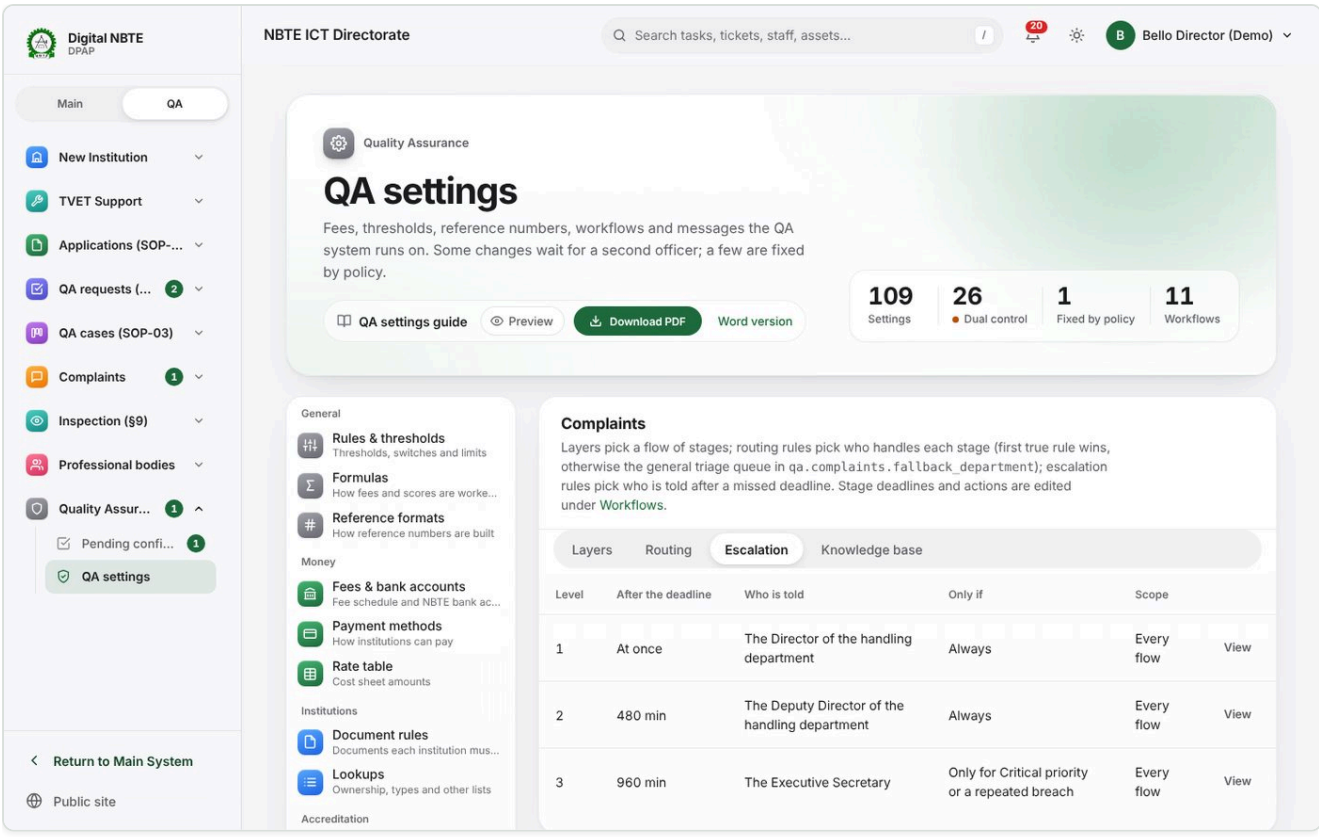
3.13 Complaints

In plain English. How complaints from institutions, resource persons and staff are sorted, routed and escalated. *Layers pick a flow of stages; routing rules pick who handles each stage (first true rule wins, otherwise the general triage queue); escalation rules pick who is told after a missed deadline.* There are four sub-sections:

- **Layers.** The kinds of complaint the raiser chooses from (ten are seeded: Payment issues, SSQ submission issues, Portal login and access, Visit scheduling, Resource-person assignment, Assessment or report dispute, Workflow delays, System problems, Permission or role, Others). Each has a name, code, optional parent (two levels at most), description shown to the raiser, who may raise it (none ticked = everyone), default priority, flow (blank = the default complaint flow) and extra questions (JSON).
- **Routing.** Rules evaluated in **order** when a complaint enters a stage: layer (blank = every layer), stage (blank = every stage with a task), an optional **condition** formula and the **target** (a department queue, a role, a named person, the institution's handling department, or the desk officer of the institution's latest QA case). The first rule whose condition is true and whose target has someone wins; if none, the complaint goes to the department in `qa.complaints.fallback_department` (ICT by default). Within a department, the least-loaded handler gets the task.
- **Escalation.** Levels with a delay after the deadline, a target (the handling department's Director, Deputy Director or Division Head, the ES, a role or a person), an *Only if* condition (always, or only for Critical priority or a repeated breach), an optional flow and stage, and the notification template to send.
- **Knowledge base.** Help articles shown while the raiser types the subject.



Complaints settings, Routing: the rules in order with layer, stage, condition and where each sends the complaint



Complaints settings, Escalation: three levels with the delay after the deadline, who is told, the Only if condition and the scope

Routing conditions may use: `priority` (`low`, `normal`, `high`, `critical`), `layer` (layer code), `parent_layer`, `raiser_type` (`institution`, `resource_person`, `staff`), `stage`, `institution_code`, `institution_zone`, `institution_state`, `handling_department` (department code) and `reopened` (times reopened). A condition is checked when you save the rule.

Who can change it. Each sub-section has its own permission: `complaints.manage_complaint_layers`, `complaints.manage_routing_rules`, `complaints.manage_escalation_rules`, `complaints.manage_knowledge_base`. Others see the tables read-only. Changes save directly.

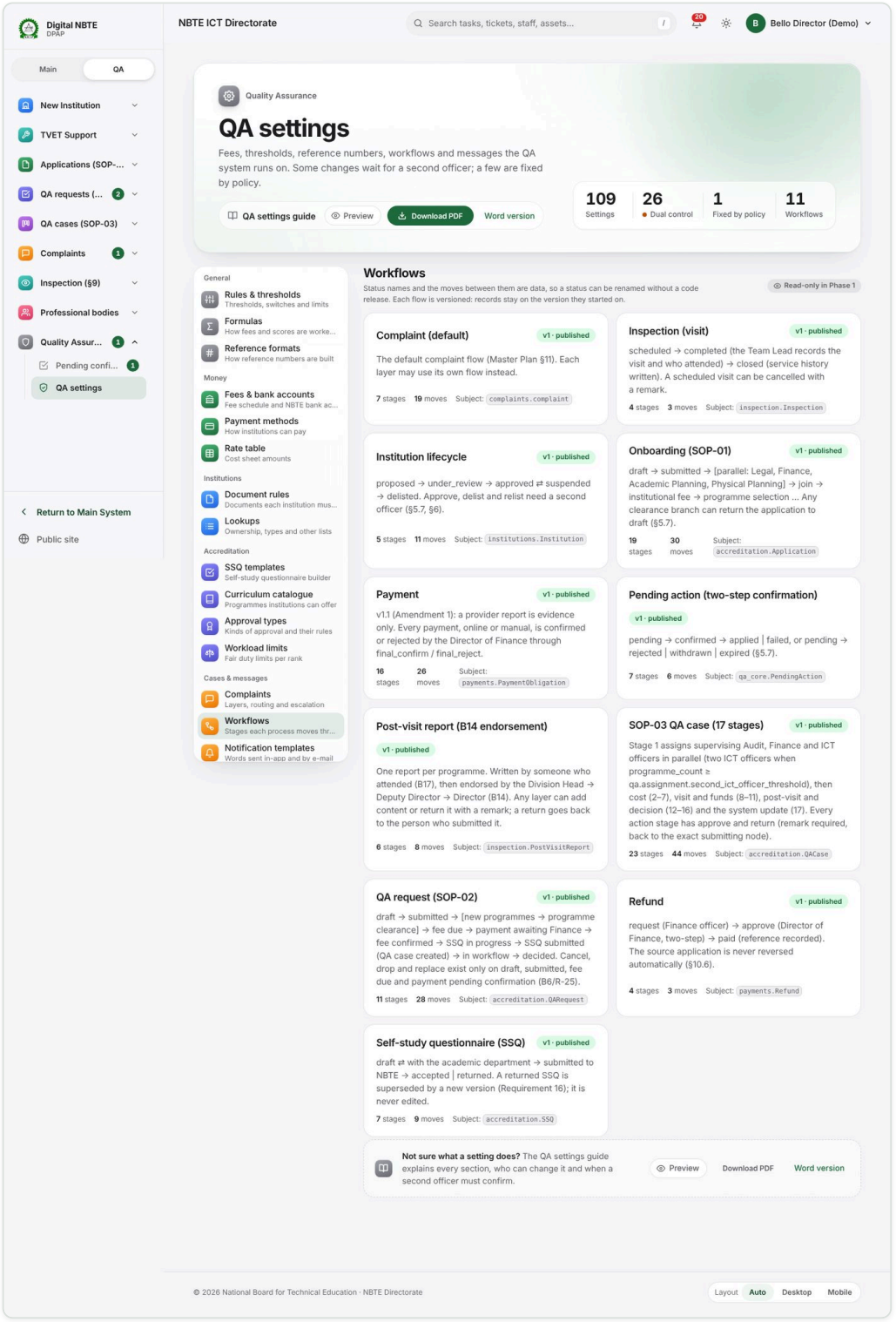
How changes take effect. At once. Routing is applied each time a complaint **enters** a stage, so a complaint already being worked keeps its handler until it moves. Escalation rules are applied by the SLA sweep (every few minutes, `qa.complaints.sla_poll_minutes`, default 5). Stage deadlines (8, 40 and 24 working hours in the default flow) belong to the workflow, not to this page.

Technically. `complaints.ComplaintLayer`, `RoutingRule`, `EscalationRule`, `KnowledgeArticle`; `complaints/services.py` (`routing_variables`, `rules_for`, `route`, `escalation_rules`, `rule_applies`); views `qa_set:complaints?section=layers|routing|escalation|kb` and `qa_set:complaints_edit`. Related settings: `qa.complaints.*`, `qa.calendar.business_hours`.

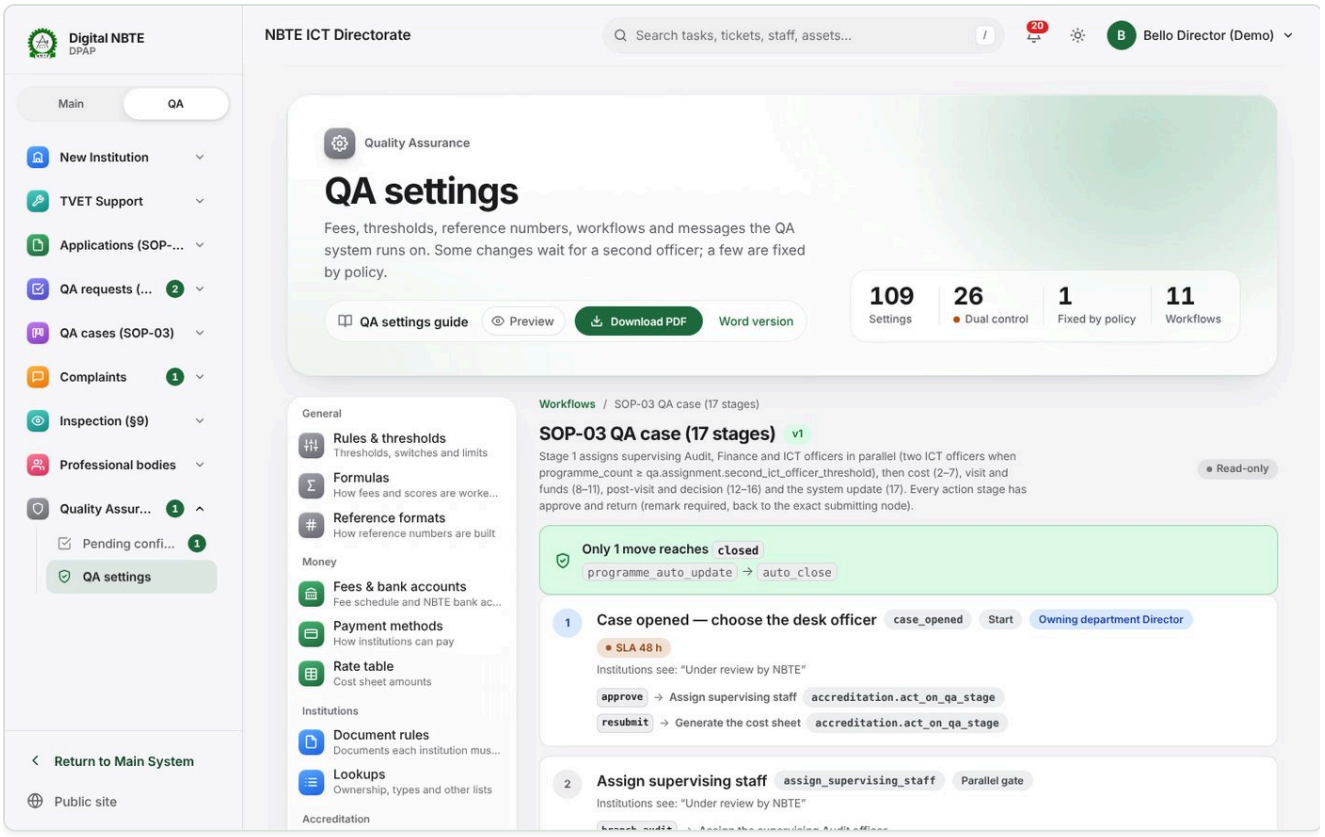
3.14 Workflows

In plain English. Every process the QA system runs is a **workflow**: a list of stages with the moves allowed between them, the actor of each stage, its deadline and any extra conditions. This tab lists all eleven: payment, pending action (two-step confirmation), institution lifecycle, onboarding (SOP-

01), refund, QA request (SOP-02), SSQ, QA case (SOP-03), inspection, post-visit report and complaint. Open one to see its stages in order, who acts, the deadline and the moves out of each stage.



Workflows: every flow with its version, published state, number of stages and moves, and the Read-only badge



One workflow (the SOP-03 QA case): its stages in order with actor, deadline and the moves out of each stage

Important: in this release the Workflows tab is read-only (the page carries the badge *Read-only in Phase 1*). There is no on-screen editor for drafts and no **Publish** button, and the Django admin also shows workflows read-only. The permissions `qa_core.manage_flows` (edit drafts) and `qa_core.publish_flow_version` (publish, Directors) exist and are granted, ready for the editor, but nothing on screen uses them yet. Earlier documents that describe *Edit a draft version; a Director publishes it* describe the intended design, not the current screens.

What you can change today that shapes a workflow without a new version:

To change...	Use
The deadline of stages that have none of their own	`qa.stage.default_sla_hours` (Rules)
When reminders are sent	`qa.stage.reminder_at_percent` (Rules)
The Finance confirmation window	`qa.payment.finance_confirmation_window_hours` (Rules)
Who handles each complaint stage, and escalation	Complaints → Routing, Escalation
The amount of a fee a workflow issues	Formulas
The wording people receive at each stage	Notification templates
Who may approve payments	`qa.payment.final_approver_role` (Rules, dual)
Who may lead a visit, eligibility, workload	Rules; Workload limits

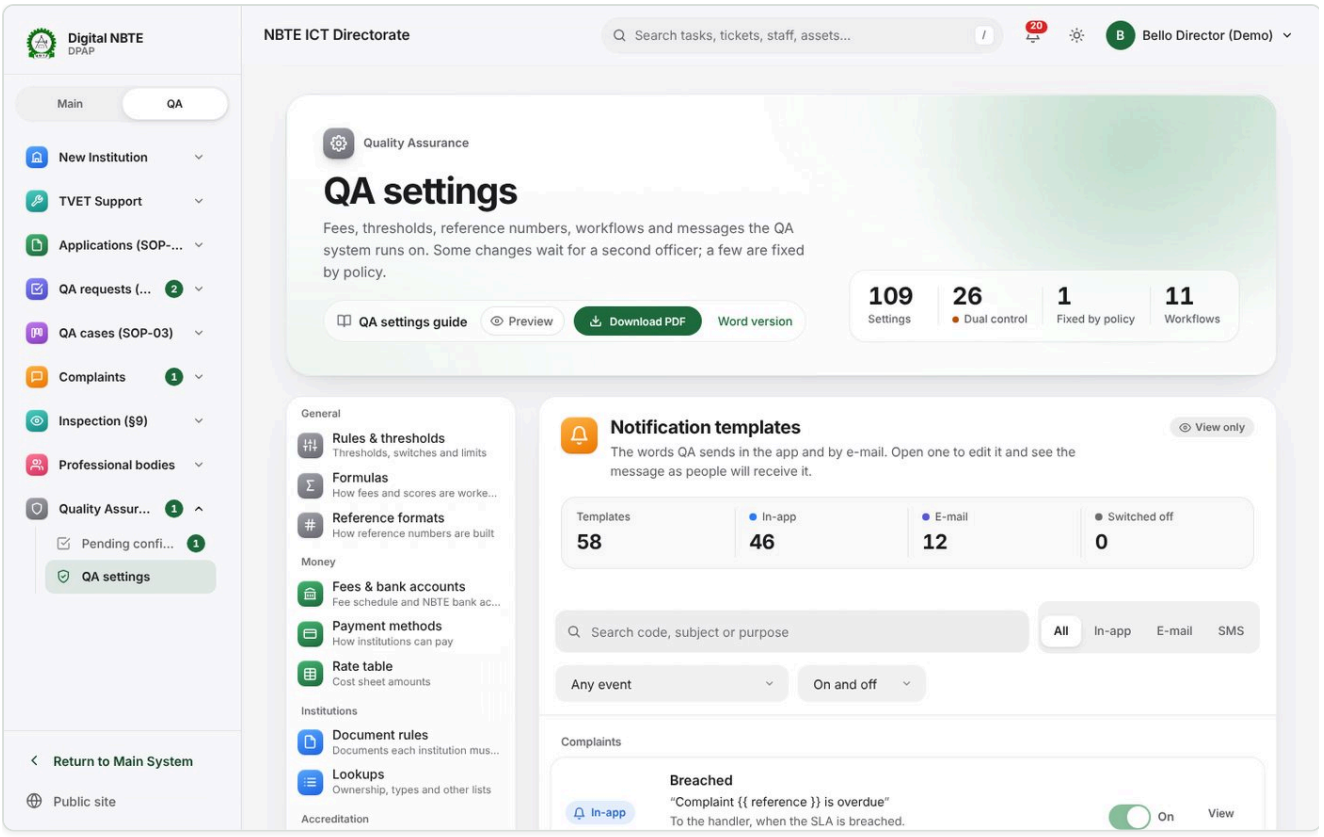
How a new version reaches the system. Stage names, deadlines (`sla_hours`), actors and moves are changed by preparing a **new version** of the flow in a release: the developers add it (for example with a data migration), it is tested on a training copy, and a Director with `qa_core.publish_flow_version` signs it off before it goes live. Walkthrough W11 explains the request.

What happens to running cases. Each running record is pinned to the version it started on and finishes on that version. New records start on the newest published, active version. So after a new QA case flow is published, cases already open keep their old stages and deadlines; only cases opened afterwards follow the new ones.

Technically. `qa_core.FlowDefinition` (`code`, `version`, `is_active`, `published_at`), `FlowStage` (`sla_hours`, `sla_business_hours`, `actor_kind`, `creates_task`, `pauses_sla`…), `FlowTransition` (`required_permission`, `required_role_setting`, `condition`, `requires_remark`, `requires_confirmation`, `effects`). `qa_core.flow.current_version(code)` picks the highest published active version; `FlowInstance.flow_version` pins a running record. `seed_startup` creates version 1 of each flow only when the code does not exist (*published flows are edited by versioning, never re-seeded*). Views `qa:settings_workflows`, `qa:workflow_detail`.

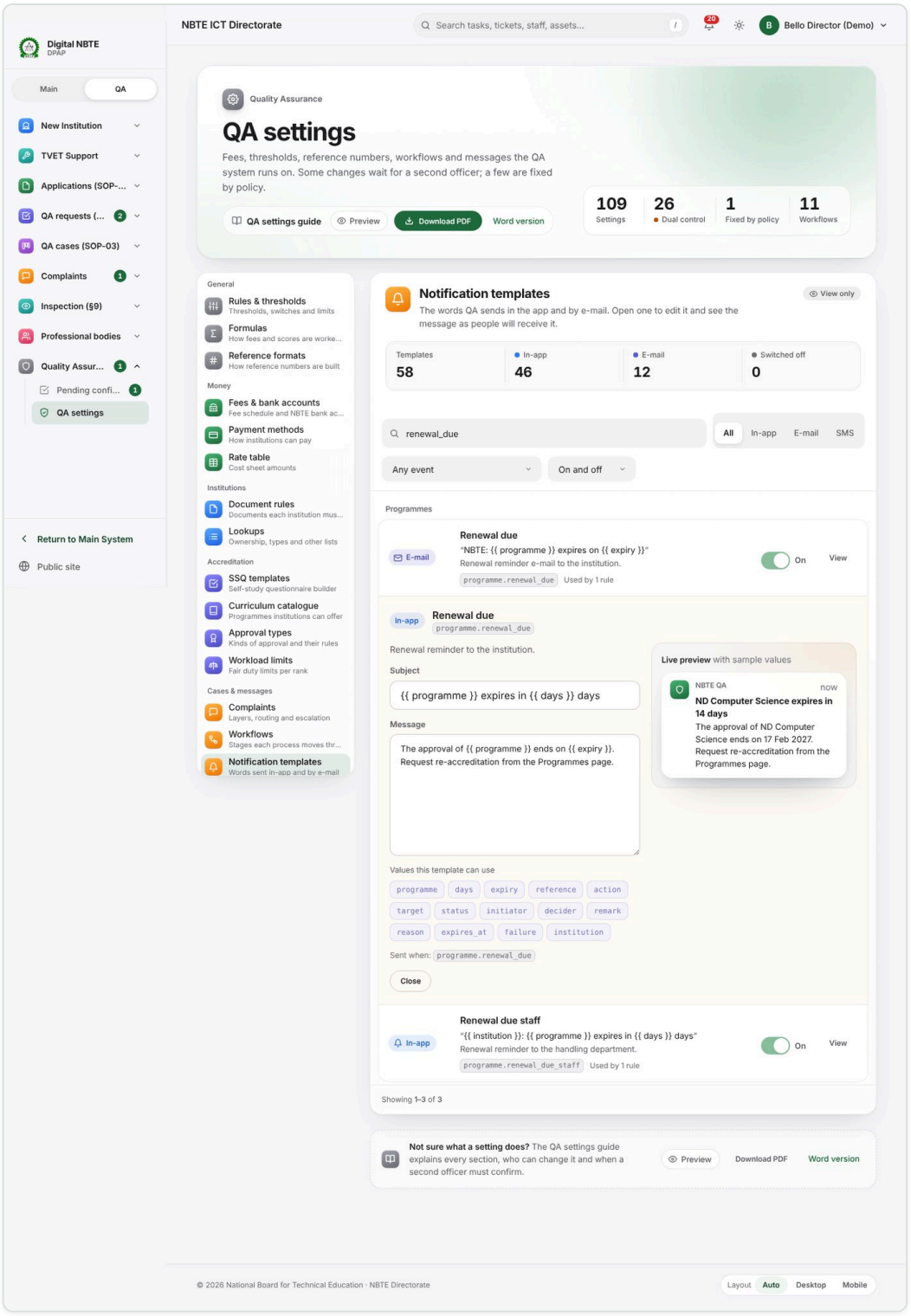
3.15 Notification templates

In plain English. The words of every in-app message and e-mail the QA system sends: to institutions (bills, decisions, renewal reminders, queries, complaints), to staff (tasks, escalations) and to confirmers (*Confirmation needed*). The page opens with four figures (templates, in-app, e-mail, switched off) and filters by text, channel, event and on/off. Templates are grouped by the part of the system that sends them.



Notification templates: counts of templates, in-app, e-mail and switched off; search, channel, event and on/off filters; templates grouped by the part of the system that sends them

Open one to see its **subject** and **message**, the **value chips** (press one to insert its placeholder, such as `{{ programme }}`, where your cursor is; the ones this template already uses come first), **Sent when** (the events that send it) and a **Live preview** that fills the placeholders with sample values and updates as you type.



A notification template open: subject and message, the value chips, Sent when, and the live preview with sample values

Templates use simple placeholders `{{ name }}` and may use `{% if name %}…{% endif %}` and filters such as `{{ status|lower }}`. E-mail messages may contain simple HTML (`<p>`, `<strong>`, `<em>`); in-app messages are shown as plain text.

Who can change it. Edit and switch on/off: `qa_core.manage_notification_templates` (QA Settings Admin). Everyone else who can open QA settings sees the editor read-only (**Close**), including Directors.

How changes take effect. At once, for messages sent after you save. A template switched off is not sent at all. Which events send which template (the *rules*) are not edited on this page.

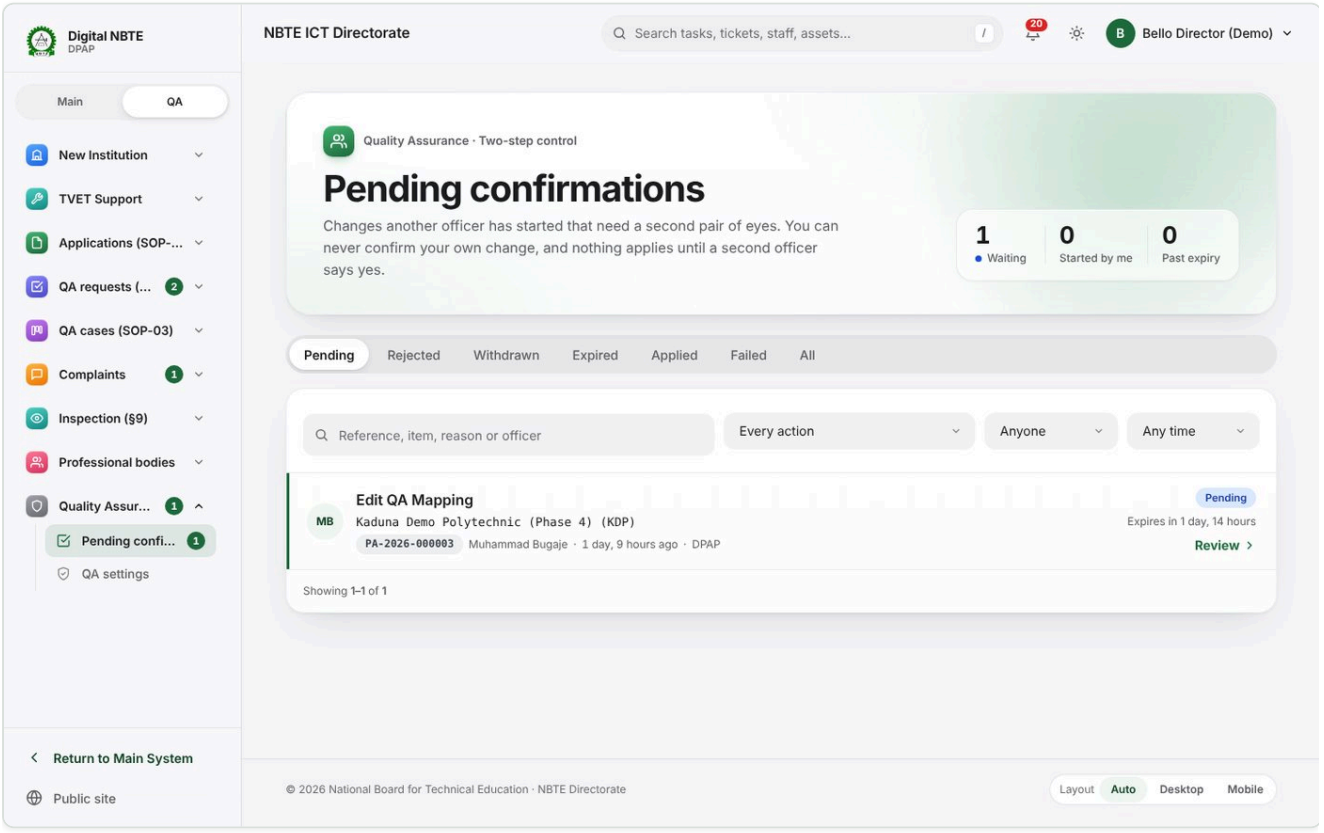
What it touches. Every message the system sends from the QA side. A placeholder the event does not supply is shown as nothing, not as an error, so a misspelt placeholder silently leaves a gap.

Technically. `qa_core.NotificationTemplate` (`code`, `channel` in_app/email, `subject`, `body`, `is_active`) and `NotificationRule` (`event_code`, `recipient_kind`, `template`); `qa_core/notify.py` renders in a sandbox (no file access, no tag libraries, plain values only, unknown variables render as empty). Views `qa:settings_templates`, `qa:template_edit`, `qa:template_preview` (sample values `PREVIEW_SAMPLE`), `qa:template_toggle`.

---

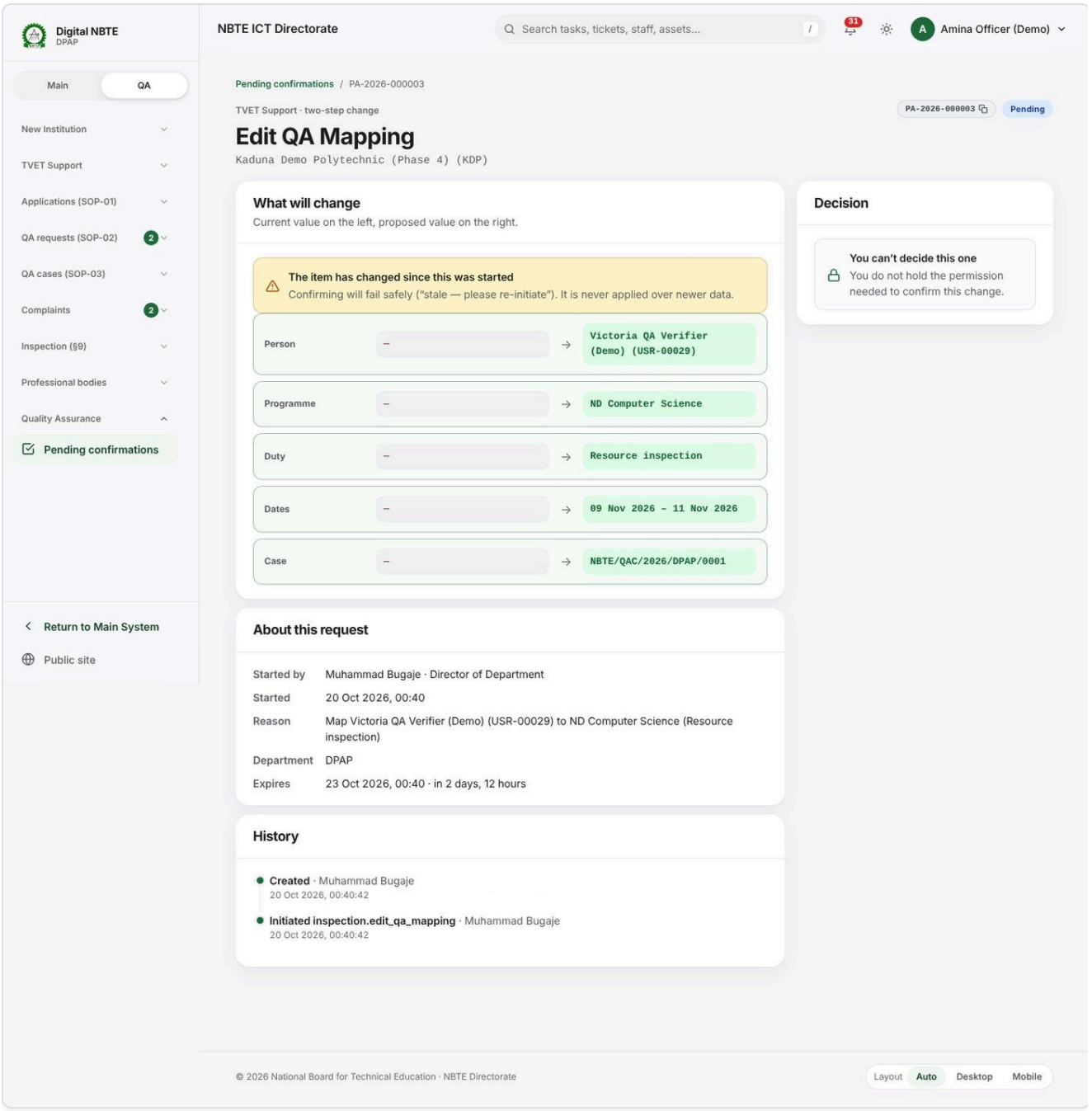
4. Pending confirmations

In plain English. The list of every change waiting for a second officer, at `/qa/pending/`. It shows counts (waiting, started by me, past expiry), tabs for **Pending**, **Rejected**, **Withdrawn**, **Expired**, **Applied**, **Failed** and **All**, and filters by action, text (reference, item, reason, officer), *mine / others* and age (24 hours, 3 days, 7 days). Pending items are listed with the soonest to expire first.



Pending confirmations: waiting, started by me and past expiry counts, tabs Pending, Rejected, Withdrawn, Expired, Applied, Failed, All, and the items waiting

Opening an item shows **What will change** (the current value on the left, the proposed value on the right), who started it, when and why, when it expires, its history and the **Decision** panel.



A pending change: What will change shows the current value on the left and the proposed value on the right; About this request; History; and the Decision panel

You are	You can
The officer who started it	Read it; **Withdraw** it while it is pending. You cannot confirm it
An eligible confirmer	**Confirm** (an optional remark) or **Reject** (a reason of at least three characters is required)
Anyone else who can see it	Read it; the Decision panel says why you cannot decide (for example *This change belongs to another department*)

After **Confirm**, the change is applied at once and the item shows *Applied*; if it could not be applied it shows *Failed* with the reason (for example *Stale…*), and nothing changed. A yellow warning appears on the item before you confirm if the record has already changed since it was started.

Who sees what. Your own items always; items in your department that you could confirm (or all items in your department with `qa_core.view_pending_actions`); everything for superusers and holders of *access all departments* or *confirm any department*.

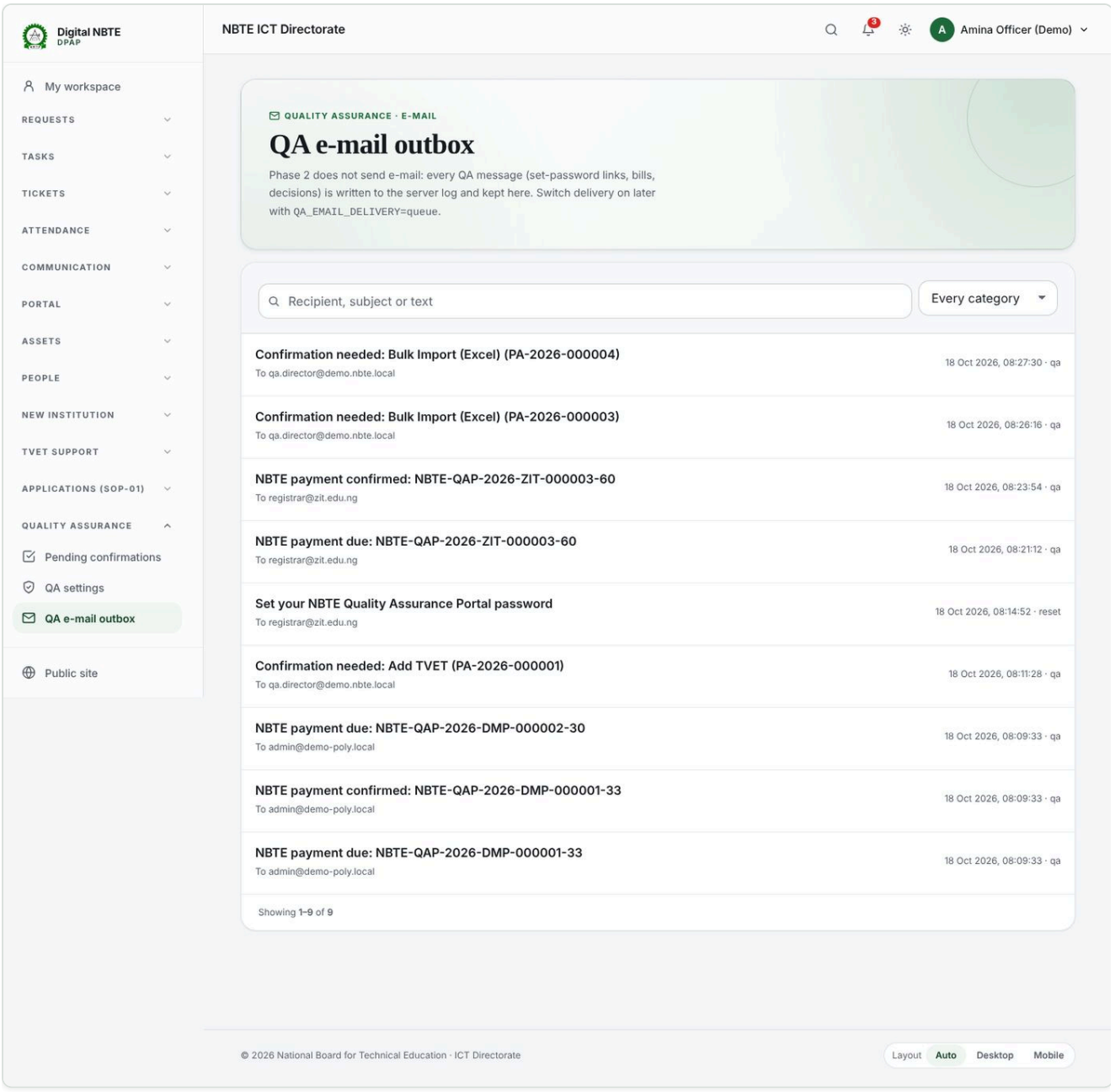
Notifications. *Confirmation needed* (in-app and e-mail) to every eligible confirmer when an item is created; *{reference}: {status}* to the initiator when it is decided; *{reference} expired* when it expires. All three are notification templates (family *Two-step changes*).

Technically. `qa_core.PendingAction` (`reference`, `policy`, `payload`, `before_snapshot`, `target_version`, `status`, `scope_department`, `expires_at`); `qa_core/actions.py` (`initiate`, `confirm`, `reject`, `withdraw`, `expire_overdue`, `visible_to`, `can_decide`). Expiry runs every 15 minutes (Celery). Views `qa:pending_list`, `qa:pending_detail`, `qa:pending_confirm`, `qa:pending_reject`, `qa:pending_withdraw`.

---

5. The e-mail outbox

In plain English. In development, training and testing, the QA system does not send e-mail. It writes every message it would have sent to the **outbox** at `/qa/outbox/`, so you can read exactly what an institution or officer would have received: bills, payment confirmations, decisions, set-password links, *Confirmation needed* messages and so on. Search by text or recipient, and filter by category and period (1, 7 or 30 days).



QA e-mail outbox: every e-mail the QA system would have sent, with recipient, subject, category and time

Because it contains one-time set-password links, only superusers, holders of the oversight board permission and QA settings administrators (`qa_core.edit_qa_settings`) can open it.

On the **live system**, e-mail is handed to the normal e-mail service instead and **is not written to the outbox**, so use the outbox for testing on a training copy, not for checking live e-mail.

Technically. `qa_core.mail.send()`; `settings.QA_EMAIL_DELIVERY` = `console` (default: logged and stored as `qa_core.OutboundEmail`) or `queue` (handed to `core.email_utils.send_templated_email`). Access check `qa_core.permissions.can_read_outbox`. View `qa:outbox`.

---

6. Walkthroughs with worked examples

Each walkthrough names the permission you need. Try it on a training copy first (section 7).

W1. Change a deadline threshold

Example: institutions should have 10 days, not 14, to answer a query.

You need: `qa_core.edit_qa_settings`.

1. Open **QA settings → Rules & thresholds**.
2. Type `query` in the search box. The rule **Default due days** (`qa.query.default_due_days`, *Default number of days an institution has to answer a query*) shows **14** and the badge **Direct**.
3. Press **Edit**, type `10` and press **Save**. The message *Saved qa.query.default_due_days* appears.
4. Press **History** on the rule: your change is listed with your name and the time.

What happens next. The next query an officer raises without typing its own due date is due in 10 days. Queries already sent keep their due dates. An officer can still type a different number of days on an individual query.

The same steps for other deadlines:

Deadline	Key	Default	Control	Applies to
Stage deadline where a stage has none	`qa.stage.default_sla_hours`	72 hours	Direct	Stage tasks opened afterwards
Stage reminders	`qa.stage.reminder_at_percent`	`[50, 90]`	Direct	The next reminder run, open tasks included
Finance confirmation window	`qa.payment.finance_confirmation_window_hours`	24 hours	Direct	Payments entering the Finance queue afterwards
Pending confirmation expiry	`qa.settings.pending_action_expiry_hours`	72 hours	**Dual**	Pending items started afterwards
Slip validity	`qa.payment.slip_validity_days`	30 days	Direct	Slips generated afterwards
Bill due date	`qa.payment.due_days`	30 days	Direct	Bills issued afterwards (unless the fee schedule sets its own)
Complaint auto-close	`qa.complaints.auto_close_days`	5 days	Direct	The next sweep

For a dual-control deadline such as `qa.settings.pending_action_expiry_hours`, step 3 becomes: enter the value and a reason, press **Send for confirmation**, and ask your Director to confirm it (W2, steps 8–12).

W2. Change the programme processing fee to charge per additional stream, and confirm it as a second officer

Example: NBTE decides that each stream beyond the first in a programme costs ₦250,000, and twice that for private institutions.

You need: to propose, `qa_core.edit_qa_settings`; to confirm, `qa_core.edit_qa_dual_control_settings`, rank 50 or more, same department (in practice your Director).

Understand the current formula first. The installed formula already has a place for an extra-stream charge:

```
programmes * (2000000 if ownership == 'private' else 1000000) + max(streams - programmes, 0)
* cfg('qa.fee.programme.per_additional_stream')
```

`max(streams - programmes, 0)` is the number of streams beyond one per programme, and `qa.fee.programme.per_additional_stream` is ₦0 until NBTE sets it. So there are two ways to make this change.

Option A — the same rate for everyone: change one amount.

1. **Rules & thresholds**, search `per_additional_stream`.
2. On **`qa.fee.programme.per_additional_stream`** (Dual control) press **Edit**, enter `250000`, give the reason (for example *Fees memo NBTE/FIN/2026/14 of 12 Oct 2026*) and press **Send for confirmation**. The message *Sent for confirmation — PA-2026-…* appears and the rule shows *PA-… awaiting confirmation*.
3. Go to step 8.

Option B — a different rate by ownership: change the formula.

1. **Formulas**, find **`qa.fee.programme_processing`** (search `programme_processing`).
2. In **Propose a new formula**, replace the text with:

   ```
   programmes * (2000000 if ownership == 'private' else 1000000) + max(streams -
   programmes, 0) * cfg('qa.fee.programme.per_additional_stream') * (2 if ownership ==
   'private' else 1)
   ```

3. **Test it before sending.** In **Test with sample values** choose `ownership` = `private` and type `institution_type` = `polytechnic`, `programmes` = `2`, `new_programmes` = `0`, `streams` = `4`, `max_streams` = `3`. Press **Run test**. With the per-stream amount still at ₦0 the result is **4,000,000.00** and the line says *settings read:* *`qa.fee.programme.per_additional_stream` = 0*. That tells you the formula is correct and that the amount must be set too (Option A, steps 1–2).

4. Type the reason, press **Send for confirmation**. The card says *PA-… is waiting for a second officer. The formula above stays in force until it is confirmed.*
5. Propose the ₦250,000 amount as in Option A. You now have two pending items; they can be confirmed in either order, but bills issued between the two confirmations use whichever is in force.

What the fee will be once both are confirmed (programmes = 2, streams = 4, so 2 extra streams):

Ownership	Calculation	Fee
Private	2 × 2,000,000 + 2 × 250,000 × 2	₦5,000,000
Federal or state	2 × 1,000,000 + 2 × 250,000 × 1	₦2,500,000
Private, one stream per programme (streams = 2)	2 × 2,000,000 + 0	₦4,000,000 (unchanged)

Confirming it (the second officer, for example the Director of DPAP).

8. You receive *Confirmation needed: Change a dual-control QA setting* in the system and by e-mail.
9. Open **Pending confirmations** (`/qa/pending/`). The item shows the reference, *Change a dual-control QA setting* and the key.
10. Open it. **What will change** shows the current formula (or value) on the left and the proposed one on the right, with the reason. Check the reason against the approved memo. If you want to check the result, open **Formulas** in another tab and run the same test.
11. Press **Confirm** (add a remark if you wish). The item shows *PA-… confirmed and applied*. To refuse, press **Reject** and give the reason; nothing changes.
12. The officer who proposed it receives *PA-…: Applied*.

What happens next. From the next programme processing bill, the new formula applies. The portal's live estimate shows the extra-stream charge as institutions choose streams. Bills already issued keep their amounts (each bill stores its formula, inputs and result).

If something goes wrong. *You started this change, so a different officer must confirm it*: ask another Director. *This change belongs to another department*: the confirmer must be in the department of the officer who started it, or hold *confirm any department*. *Stale…*: the setting was changed after you started; start again.

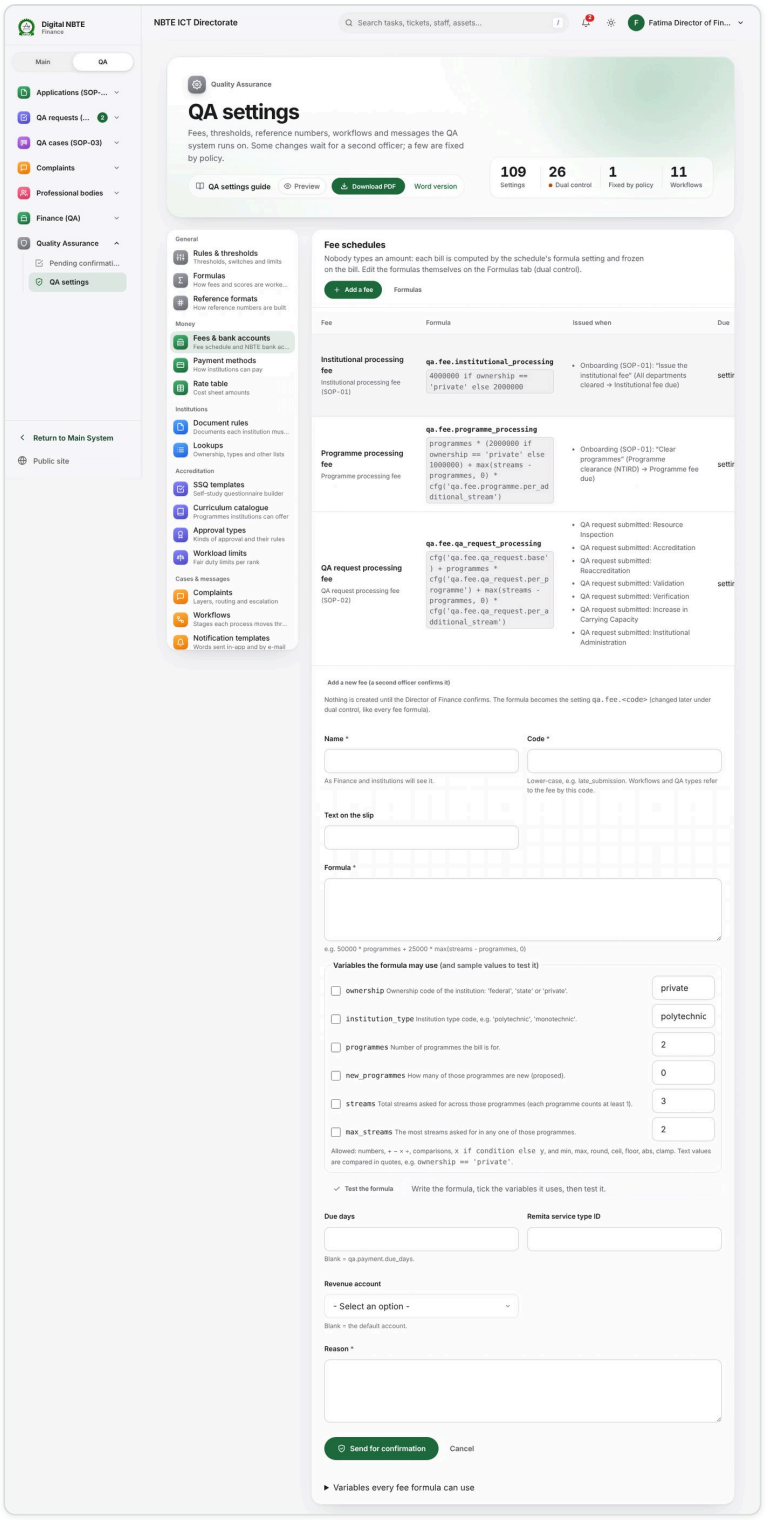
W3. Add a brand-new fee, test it, and issue a bill

Example: a late submission fee of ₦50,000 per programme plus ₦25,000 for each extra stream.

You need: to propose the fee and the bill, `payments.view_obligation` (a QA Finance Officer); to confirm both, the Director of Finance (`payments.manage_fee_schedule` for the fee, `payments.final_confirm_payment` for the bill, rank 100, Finance department).

Part 1 — propose the fee.

1. Open **QA settings → Fees & bank accounts** and press **Add a fee**.
2. Fill in:
   - **Name:** `Late submission fee` (as Finance and institutions will see it);
   - **Code:** `late_submission` (3 to 51 lower-case letters, digits or underscores, starting with a letter; it cannot be changed later);
   - **Text on the slip:** `Late submission fee (SOP-02)`;
   - **Formula:** `50000 * programmes + 25000 * max(streams - programmes, 0)`;
   - **Variables:** tick `programmes` and `streams` (every variable the formula uses must be ticked, or the test says *Unknown name*);
   - **Sample values:** `programmes` = `2`, `streams` = `3` (the others are ignored by this formula);
   - **Due days:** blank (uses `qa.payment.due_days`, 30) or a number from 1 to 365;
   - **Remita service type ID** and **account:** blank uses the defaults;
   - **Reason:** for example *Approved by management, memo of 15 Oct 2026*.
3. Press **Test the formula**. The result is **₦125,000.00** (2 × 50,000 + 1 × 25,000). If the formula gives zero or less, the test says so: a bill must be more than zero.
4. Press **Send for confirmation**. The message *PA-… waiting for a second officer* appears; the item is listed under **Waiting for confirmation** on the same page.



Fees and bank accounts with the Add a fee form open: name, code, text on the slip, formula, the variables with sample values, Test the formula, due days, Remita service type, account and reason

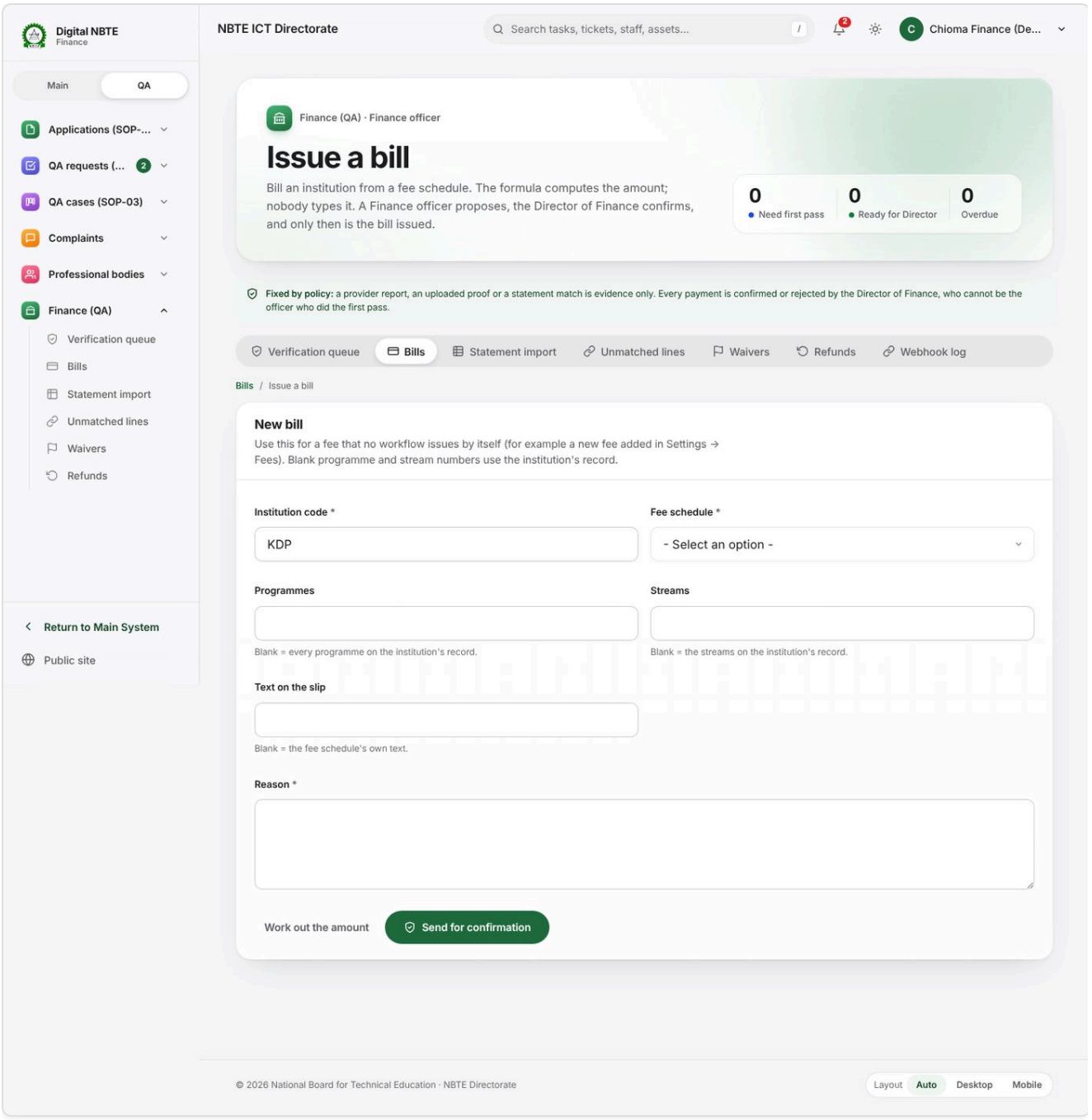
Part 2 — the Director of Finance confirms the fee.

5. The Director of Finance opens **Pending confirmations**, opens *Add Fee Schedule — New fee: Late submission fee (late_submission)*. The page lists the fee, its formula setting (`qa.fee.late_submission`), the formula, the variables, the due days, the Remita service type and the **test with the sample inputs** (₦125,000.00).

6. **Confirm.** The system creates the dual-control formula setting `qa.fee.late_submission` (visible on **Formulas**) and the fee schedule `late_submission` (visible on **Fees**, marked *Not issued automatically — Finance issues it from an institution's bills page*).

Part 3 — issue a bill.

7. A Finance officer opens **Finance → Bills → Issue a bill** (`/qa/finance/obligations/issue/`), or **Issue a bill** on an institution's Payments tab.
8. Enter the **institution code** (for example `DMP`), choose **Late submission fee**, and either leave **programmes** and **streams** blank (the institution's record is used: every programme not dropped, with its streams) or type them. Add a reason.
9. Press **Work out the amount** to see the amount, inputs and formula. Then **Send for confirmation**.
10. The Director of Finance confirms *Issue a Bill* on **Pending confirmations**. The bill is created with a payment reference such as `NBTE-QAP-2026-DMP-000123-45`, its formula snapshot is frozen, the slip is published (if a real bank account is confirmed), and the institution is notified (*payment.issued*, in-app and e-mail).



Issue a bill: institution code, fee schedule, programmes and streams, Work out the amount and Send for confirmation

What happens next. The fee stays *issued by hand* until a workflow step (`create_payment_obligation:late_submission`) or a QA type is pointed at it, which needs a release. To change its amount later, change `qa.fee.late_submission` on **Formulas** (dual control, W2).

W4. Edit an approval type's duration

Example: interim accreditation should last 18 months instead of 12.

You need: `accreditation.manage_approval_types`.

1. Open **QA settings → Approval types**.

2. On **Interim accreditation**, change the duration from `12` to `18` and press **Save**. The message *Interim accreditation: 18 months* appears.

What it does to expiry dates. Expiry = commencement date + duration.

Programme	Commencement	Applied at stage 17	Expiry
A — applied last month	1 Sep 2026	Before your change	1 Sep 2027 (kept)
B — ES decision recorded, stage 17 not yet run	1 Oct 2026	After your change	**1 Apr 2028**
C — decided next month	1 Dec 2026	After your change	1 Jun 2028

The re-accreditation window opens `qa.programme.reassessment_window_months` (6) months before each expiry, and renewal reminders follow `qa.programme.renewal_reminder_days`. Outcome letters print the expiry worked out at stage 17. Programme A's date is not changed; if it must be, that is a separate decision recorded on the programme.

W5. Set workload limits for a rank

Example: Directors should make at most 4 visits a year and hold no more than 2 SSQ reviews at once; resource persons at most 1 SSQ review at once.

You need: `qa_core.edit_qa_settings`.

1. Open **QA settings → Workload limits**. The **Everyone else** row is blank: the general rules show in grey (3 SSQ reviews at once, 6 visits a year, same institution once in 24 months).
2. Press **Add a rank**. In the new row choose **DIRECTOR (designation)**, type `2` under *SSQ reviews at once* and `4` under *Visits a year*. Leave other cells blank. (The list shows each code once, designations first; because the Director designation and the Director of Department role share the code `director`, this one row covers both.)
3. Press **Add a rank** again, choose **Resource persons (external)**, type `1` under *SSQ reviews at once*.
4. Type a reason and press **Save limits**. The message *Workload limits saved — They apply to the next assignment* appears.

What happens next. A Director who already has 4 visits this year is refused for a fifth visit team with a message naming the limit (*qa.workload.limits · director*). The finder shows each person's current load. Assignments already made are not undone. A row for a **designation** beats a row for a **role**, which beats a row for a kind of person.

W6. Build and publish a new SSQ template version

Example: add a question about the institution's solar back-up to the Reaccreditation SSQ.

You need: `accreditation.manage_ssq_templates` to build; `accreditation.publish_ssq_template` (a Director) to publish.

1. Open **QA settings → SSQ templates**. On **Reaccreditation** press **New version**. Version 2 is created as a copy of version 1; the builder opens.
2. Choose the section (for example *Facilities*), press **Add field**, choose **Yes / No**, and enter key `solar_backup`, label *Does the department have a solar back-up for its laboratories?*, help text and **required**. Save the field.
3. Optionally add a **File (evidence)** field tied to a document kind for a photograph or invoice.
4. Press **Preview** to see the questionnaire as institutions will. Check the message *The schema is valid*; if problems are listed, fix them.
5. Ask a Director to review the draft. The Director opens the same draft and presses **Publish**. The message *Version 2 published — new SSQs use it* appears; version 1 becomes *retired*.

What happens next. SSQs opened from now on (when a Reaccreditation request's fee is confirmed) use version 2. SSQs already open stay on version 1, and submitted ones keep their snapshot. If you change your mind before publishing, **Discard draft**.

W7. Add catalogue items

Example: Computer Science needs 10 network cable testers and a server rack.

You need: `accreditation.manage_catalogue`.

1. Open **QA settings → Curriculum catalogue**.
2. Under **Add an item**, type programme name `Computer Science` (a programme name makes the item apply to that programme only; leave the name blank and choose a category to apply it to every programme in the category), choose **Equipment**, name `Network cable tester`, quantity `10`, unit `no.`; press **Add item**.
3. Add `Server rack (42U)`, quantity `1`.
4. To change an existing line, press **Change…** on it, enter the new quantity and press **Save as version n**. To remove one, **Retire item**.

What happens next. Catalogue questions in SSQs for Computer Science list the new items at once. SSQs already submitted keep the old list.

W8. Update the rate table

Example: Finance has confirmed the duty tour allowance for officers at ₦55,000 a day, and DPAP uses ₦80,000 for its team leads.

You need: `accreditation.manage_rate_table`.

1. Open **QA settings → Rate table**.
2. On **Team member (officer)** (`dta-officer`) enter `55000` and press **Save**. The message *Team member (officer): ₦55,000.00* appears.
3. For the department override, use **Add a rate**: code `dta-director`, who it is for *Team lead (Director / Deputy Director)*, kind *Duty tour allowance*, unit *Per day*, amount `80000`, department *Polytechnic and Allied Programmes Department*; press **Add rate**. DPAP's cost sheets now use ₦80,000; other departments keep the shared ₦75,000.

4. When every rate is confirmed, ask the settings administrator to set `qa.cost.rates_confirmed` to on (Rules & thresholds; dual control). Cost sheets then stop showing *provisional*.

What happens next. A 4-day visit by one officer prices at 4 × ₦55,000 = ₦220,000 on cost sheets built from now on. Lines already priced keep their amounts.

W9. Add a complaint layer with routing and escalation

Example: a new kind of complaint, "Outcome letter errors", for institutions, handled by the institution's handling department, with critical ones going straight to that department's Director, and an extra escalation to the ES after 24 working hours.

You need: `complaints.manage_complaint_layers`, `complaints.manage_routing_rules`, `complaints.manage_escalation_rules` (the QA Settings Admin role or the *Complaints Admin* group).

1. **Layer.** Open **QA settings → Complaints → Layers**, open **Add a layer** and fill in: name `Outcome letter errors` (code made from the name: `outcome-letter-errors`), description *A wrong name, programme, date or approval type on an outcome letter*, who may raise it: **Institutions**, default priority **High**, flow blank (the default complaint flow), extra questions:

   ```
   [{"key": "letter_reference", "label": "Letter reference", "type": "text", "required": true}]
   ```

   Press **Add**.

2. **Routing.** Open **Routing**, **Add a routing rule**:

   - order `56`, layer *Outcome letter errors*, stage blank, condition `priority == "critical"`, target **A role**, role *Director of Department*, department blank (the institution's handling department is used), note *Critical letter errors go to the Director*. **Add**.
   - order `57`, the same layer, no condition, target **The institution's handling department**. **Add**.

   Rules run in order; the first whose condition is true and that finds someone wins. Without these rules the complaint would go to the general triage queue (`qa.complaints.fallback_department`, ICT).

3. **Escalation.** The three seeded levels already apply to every complaint flow (level 1 at once to the Director, level 2 after 480 minutes to the Deputy Director, level 3 after 960 minutes to the ES only for Critical or repeated breaches). To add an extra level only for this kind of complaint you would scope it by flow and stage; because escalation rules are scoped by **flow**, not by layer, a level for one layer only needs that layer to use its own flow. For this example, simply check that the existing levels suit it.

4. **Knowledge base (optional).** Add an article *Checking your outcome letter* linked to the layer, so institutions see it while typing.

What happens next. Institutions see the new layer at once when raising a complaint. A complaint is routed when it enters each stage; a missed working-hour deadline escalates through the levels at the next sweep.

W10. Draft and publish a new workflow version

Example: the Director wants the SOP-03 stage "Generate the cost" to have 96 hours instead of 72, and a new stage added.

In this release there is no on-screen workflow editor (3.14). Do this instead:

1. **Check whether a setting is enough.** If the stage has no deadline of its own, `qa.stage.default_sla_hours` changes it (W1). Reminders, Finance windows, routing and wording are also settings.
2. **Write the change request.** Open **QA settings → Workflows**, open the flow (for example *QA case*), and note the stage codes, the current deadlines and moves. Write down exactly what should change (stage, new deadline, new stage name, who acts, which moves) and why.
3. **The developers prepare a new version** of the flow (version *n + 1*) in a release, and install it on a training copy.
4. **Test on the training copy.** Start a new record and walk it through the changed stages; check that tasks open with the new deadlines and the right people.
5. **A Director approves publication.** The Director (holder of `qa_core.publish_flow_version`) signs off the tested version; ICT installs the release on the live system.

What happens to running cases. Cases, requests and complaints already running stay on the version they started with, with their old stages and deadlines, until they finish. Only records started after publication follow the new version. If a change must also reach running cases, that is a separate data correction for the developers, and should be rare.

W11. Edit a notification template with placeholders and preview

Example: make the renewal reminder e-mail mention the re-accreditation window.

You need: `qa_core.manage_notification_templates`.

1. Open **QA settings → Notification templates**, search `renewal_due`, filter channel **E-mail**.
2. Open **Programme renewal due** (`programme.renewal_due`, E-mail). **Sent when:** `programme.renewal_due`.
3. Edit the message. Place the cursor where you want a value and press a chip to insert it, for example:

```
<p>The approval of <strong>{{ programme }}</strong> ends on <strong>{{ expiry }}
</strong> ({{ days }} days from now).</p>
<p>You may now request re-accreditation from the <em>Programmes</em> page of the NBTE
Quality Assurance Portal. Requests should be made at least six months before expiry.</p>
```

4. Watch the **Live preview**: it fills the placeholders with sample values (for example *ND Computer Science*, *17 Feb 2027*, *14*). If a part is blank in the preview, the placeholder is misspelt or not available.
5. Press **Save template**. The message *Saved template programme.renewal_due* appears.

6. To stop a message entirely, switch the template **off** from the list.

What happens next. The next renewal reminder sent (by the daily expiry job) uses the new words. On a training copy you can read it in the outbox (section 5). Template errors (for example an unclosed `{% if %}`) are refused on save with the reason.

W12. Change a reference format

Example: add a check digit to complaint references, so that a mistyped complaint number can be spotted: `NBTE-CMP-{YEAR}-{SEQ:6}` *becomes* `NBTE-CMP-{YEAR}-{SEQ:6}-{CHK97}`.

You need: to propose, `qa_core.manage_reference_formats`; to confirm, a same-department officer with `qa_core.edit_qa_dual_control_settings`, rank 50 or more.

1. Open **QA settings → Reference formats** and press the edit button on **Complaint**.
2. Change the pattern to `NBTE-CMP-{YEAR}-{SEQ:6}-{CHK97}` and the **check digit** to *ISO 7064 mod 97-10*. Keep reset *Yearly* and scope *One series for everything*. Give a reason and press **Send for confirmation**.
3. If the pattern is not valid you see why (for example *The pattern must contain exactly one {SEQ}*, *The check digit must come after {SEQ}*, *{CHK97} needs check_digit = mod97*, *The pattern contains characters outside the allowed set*).
4. The Director confirms it on **Pending confirmations**.

What happens next. The next complaint gets, for example, `NBTE-CMP-2026-000042-17`; earlier complaints keep their references. The number continues from where it was. Use **Check a payment reference** to test any reference with a mod 97 check.

W13. Switch portal self-registration off

Example: NBTE wants new institutions to register only through the QA Directorate during an audit.

You need: `qa_core.edit_qa_settings`.

1. Open **QA settings → Rules & thresholds** (no filter). At the top is the card **Institution self-registration**.
2. First set **Registration contact email** (`qa.portal.registration_contact_email`) and **Registration contact phone** (`qa.portal.registration_contact_phone`) with **Edit → Save**, for example `qa@nbte.gov.ng` and `+234 …`. Blank uses the site's public contact details.
3. Flip **Self registration enabled** (`qa.portal.self_registration_enabled`) to **Off**. It saves at once; the card's line changes to *Registration is managed by NBTE*.

What happens next. The portal hides the registration form at once and asks visitors to contact the QA Directorate at the e-mail and phone you set. NBTE staff can still register institutions with **Add TVET** (internal, two officers). Institutions already registered are unaffected. The downloadable institution guides print the contact details when they are generated, so ask ICT to run `manage.py generate_institution_guides` after changing the contacts. Switch it back **On** to reopen registration.

W14. Adjust a capacity constant safely

Example: NBTE's worksheet uses 0.75 for classroom use, not 0.8.

1. **Rules & thresholds**, search `classroom.use_factor`, **Edit**, `0.75`, **Save** (direct).
2. Assessments run from now on use 0.75: in the master guide's example (four 40-seat rooms, 20 contact hours), standard hours become 8 × 5 × 15 × 0.75 = 450, additional streams (450 × 4) ÷ 300 = 6.0, so 6 streams supportable (unchanged here; smaller rooms would show the difference).
3. Stored assessments are not changed. To apply the new constant to a case, an officer re-runs its assessment (a new version is kept beside the old one).

The capacity formula itself (`qa.capacity.additional_streams`) is dual control and cannot use `cfg()`.

---

7. Testing your change safely

In plain English. A settings change can reach every institution within minutes. Before changing the live system, prove the change on a copy and use the system's own test tools.

1. **Use a training copy.** ICT starts a copy with demonstration data (master guide, *Trying it without touching real data*; demo accounts in Part E). Make the change there first, walk one record through, and only then repeat it on the live system. Never test by changing live values and changing them back: bills, tasks and references created in between keep the test values.
2. **Use the formula Test box.** Every formula card has **Test with sample values**, which tests the text *in the editor* without saving. Try at least: the smallest case (one programme, one stream), a typical case, a large case, each ownership (`private`, `federal`, `state`), and any edge your formula has (for example `streams` equal to `programmes`). Check the *settings read* line shows the values you expect from `cfg()`. **Add a fee** has its own **Test the formula**, and its test is repeated on the confirmation page.
3. **Use Work out the amount** on **Issue a bill** to see the amount for a real institution before anything is sent.
4. **Use the template preview.** The live preview fills every placeholder with sample values. A blank where a value should be means a misspelt or unavailable placeholder.
5. **Use the SSQ Preview** before publishing a template, and check *The schema is valid*.
6. **Use the outbox** on the training copy to read the e-mails your change produces (bills, confirmations, reminders) exactly as they would be sent.
7. **Use Pending confirmations as a review step.** The confirmer sees before and after side by side with your reason. Put the memo reference in the reason, so the confirmer can check it. If a change is wrong, the confirmer rejects it and nothing happens; if you spot a mistake first, **Withdraw** it.
8. **Check History afterwards.** Each rule's **History** and the audit log show what changed, when and by whom.
9. **Change one thing at a time,** and note the old value before you change it, so you can put it back.

On the live system, e-mail is sent for real and the outbox stays empty; test notification changes on the training copy.

---

8. The most important settings keys

Defaults are those installed by `seed_startup` (`qa_core/settings_catalogue.py`). *Dual* = needs a second officer. All are edited on **Rules & thresholds** unless stated.

Fees and payments

Key	Default	Type	Control	Meaning
`qa.fee.institutional_processing`	`4000000 if ownership == 'private' else 2000000`	formula	Dual	SOP-01 institutional processing fee (Formulas tab)
`qa.fee.programme_processing`	`programmes * (2000000 if ownership == 'private' else 1000000) + max(streams - programmes, 0) * cfg('qa.fee.programme.per_additional_stream')`	formula	Dual	Programme processing fee (Formulas tab)
`qa.fee.qa_request_processing`	`cfg('qa.fee.qa_request.base') + programmes * cfg('qa.fee.qa_request.per_programme') + max(streams - programmes, 0) * cfg('qa.fee.qa_request.per_additional_stream')`	formula	Dual	SOP-02 fee (Formulas tab)
`qa.fee.qa_request_processing.confirmed`	false	bool	Dual	While false, SOP-02 bills cannot be issued
`qa.fee.qa_request.base`	0	decimal	Dual	SOP-02 base amount (₦)
`qa.fee.qa_request.per_programme`	0	decimal	Dual	SOP-02 amount per programme (₦)

Key	Default	Type	Control	Meaning
`qa.fee.programme.per_additional_stream`	0	decimal	Dual	₦ per stream beyond one per programme (programme fee)
`qa.fee.qa_request.per_additional_stream`	0	decimal	Dual	₦ per stream beyond one per programme (SOP-02 fee)
`qa.payment.final_approver_role`	`qa-finance-director`	str	Dual	Role that gives the final confirm/reject on every payment
`qa.payment.receipt_required_for_provider_payments`	true	bool	Dual	Provider-reported payments need an uploaded receipt
`qa.payment.provider_confirmation_is_evidence_only`	true	bool	Fixed	A gateway report is evidence, never confirmation
`qa.payment.remita.enabled` / `qa.payment.paystack.enabled`	true in development, false in production	bool	Dual	Offer Remita / Paystack (needs server keys)
`qa.payment.bank_transfer.enabled`	true	bool	Dual	Offer bank transfer (forced on if all else is off)
`qa.payment.offline.enabled`	true	bool	Dual	Allow fee waivers
`qa.payment.method_order`	`["remita", "paystack", "bank_transfer"]`	json	Direct	Order of methods shown
`qa.payment.paystack.fee_bearer`	`institution`	str	Dual	Who bears the Paystack fee (`institution` or `nbte`)
`qa.payment.slip_validity_days`	30	int	Direct	Days a slip stays valid
`qa.payment.due_days`	30	int	Direct	Days from issue to due date

Key	Default	Type	Control	Meaning
`qa.payment.finance_confirmation_window_hours`	24	int	Direct	Hours before *Payment Pending Confirmation*
`qa.payment.dual_verify_threshold`	5000000	decimal	Dual	Payments above this need a second first-pass check
`qa.payment.allow_partial`	false	bool	Dual	Allow part payment online
`qa.payment.apply_credit_automatically`	false	bool	Dual	Apply held credit to the next bill
`qa.payment.proof_extensions` / `qa.payment.proof_max_mb`	`["pdf", "jpg", "jpeg", "png"]` / 5	json / int	Direct	Proof-of-payment file types and size
`qa.cost.rates_confirmed`	false	bool	Dual	Finance has confirmed the rate table

Workflow, deadlines and assignment

Key	Default	Type	Control	Meaning
`qa.stage.default_sla_hours`	72	int	Direct	Deadline for stages that have none
`qa.stage.reminder_at_percent`	`[50, 90]`	json	Direct	Reminders at these percentages of the deadline
`qa.settings.pending_action_expiry_hours`	72	int	Dual	Pending confirmations expire after this
`qa.query.default_due_days`	14	int	Direct	Days to answer a query
`qa.qa_request.max_open_per_type`	1	int	Direct	Open requests of one QA type per institution
`qa.assignment.second_ict_officer_threshold`	5	int	Direct	Programmes needing two tracking ICT officers

Key	Default	Type	Control	Meaning
`qa.assignment.max_programmes_per_verifier`	5	int	Direct	Programmes per verification officer
`qa.assignment.verifier_same_discipline`	true	bool	Direct	Verifiers share the programme's discipline
`qa.workload.reviewer_cannot_verify`	true	bool	Dual	A reviewer cannot be on the visit team
`qa.visit.team_lead_roles`	`["director", "deputy-director", "zonal-director"]`	json	Direct	Who may lead a visit
`qa.matching.travel_buffer_days`	2	int	Direct	Days around a visit when a person cannot be booked
`qa.matching.same_institution_cooldown_months`	24	int	Direct	Months before the same assessor returns
`qa.calendar.business_hours`	`["08:00", "16:00"]`	json	Direct	Working hours for complaint deadlines
`qa.suspension.enabled_for`	`[]`	json	Dual	Record types a complaint may place on hold

Eligibility and workload

Key	Default	Type	Control	Meaning
`qa.resource_person.min_rank`	`director`	str	Direct	Minimum role rank for staff resource persons
`qa.resource_person.required_designations`	`["director", "deputy-director", "chief-lecturer", "principal-lecturer"]`	json	Direct	Allowed designations and ranks

Key	Default	Type	Control	Meaning
`qa.resource_person.max_active_engagements`	2	int	Direct	Simultaneous live mappings per person
`qa.resource_person.max_open_ssq_per_officer`	3	int	Direct	General limit: SSQ reviews at once
`qa.resource_person.max_visits_per_year`	6	int	Direct	General limit: visits a year
`qa.staff.ssq_grade_min` / `qa.staff.ssq_grade_max`	7 / 12	int	Direct	Grade levels for officer-level QA work
`qa.workload.limits`	`{"rows": [{"key": "default"}]}`	json	Direct	Limits per rank (Workload limits tab)
`qa.workload.staff_first`	true	bool	Direct	Staff before resource persons in the finder
`qa.workload.resource_persons_only_when_no_staff`	false	bool	Direct	Block resource persons while staff are free

Institutions, portal and access

Key	Default	Type	Control	Meaning
`qa.portal.self_registration_enabled`	true	bool	Direct	Institutions may self-register
`qa.portal.registration_contact_email` / `_phone`	blank	str	Direct	Contact shown when registration is off
`qa.portal.maintenance_message`	blank	str	Direct	Banner on every portal page while not empty
`qa.portal.staff_sign_in_enabled`	true	bool	Direct	NBTE staff may sign in to the portal
`qa.credentials.link_valid_hours`	72	int	Direct	Set-password link validity

Key	Default	Type	Control	Meaning
`qa.credentials.resend_cooldown_minutes`	30	int	Direct	Minutes between two set-password e-mails
`qa.impersonation.max_minutes`	30	int	Dual	Longest impersonation session
`qa.impersonation.allow_write`	false	bool	Dual	Allow write-mode impersonation
`qa.institution.admin_min_grade_level`	12	int	Direct	Lowest grade for an Institutional Administrator
`qa.institution.proposed_department`	`ntird`	str	Direct	Department handling proposed institutions
`qa.ownership_change.hold_open_cases`	true	bool	Direct	Hold the open application on ownership change
`qa.documents.max_upload_mb`	10	int	Direct	Largest statutory-document upload
`qa.streams.max_per_programme`	10	int	Direct	Most streams per programme
`qa.ssq.submit_roles`	`["inst-chief-executive", "inst-admin"]`	json	Direct	Institution roles that may submit an SSQ

Decisions, letters and expiry

Key	Default	Type	Control	Meaning
`qa.programme.reassessment_window_months`	6	int	Direct	Months before expiry when re-accreditation may be requested
`qa.programme.expiry_warning_days`	`[180, 90, 30]`	json	Direct	Days before expiry for institution warnings
`qa.programme.renewal_reminder_days`	`[180, 90, 30, 7]`	json	Direct	Renewal reminders to institution and department

Key	Default	Type	Control	Meaning
`qa.cqac.batch_approval_enabled`	true	bool	Direct	CQAC Chairman may approve in a batch
`qa.letters.signatory`	`{"name": "", "title": "Executive Secretary"}`	json	Dual	Name and title under the signature
`qa.letters.template`	the standard wording	json	Direct	Outcome letter wording with `{placeholders}`
`qa.letters.watermark`	`{"text": "NBTE DUPLICATE", "opacity": 0.12, "angle": 45}`	json	Direct	Watermark on the duplicate letter

Capacity engine

Key	Default	Control
`qa.capacity.hours_per_day` / `days_per_week` / `weeks_per_semester`	8 / 5 / 15	Direct
`qa.capacity.classroom.use_factor` / `qa.capacity.lab.use_factor`	0.8 / 0.75	Direct
`qa.capacity.classroom.size_factors`	`[[15, 20, 0.5], [30, 40, 1.0], [45, 60, 1.5], [60, null, 2.0]]`	Direct
`qa.capacity.lab.size_factors`	`[[15, 0.5], [30, 1.0], [45, 1.5]]`	Direct
`qa.capacity.additional_streams`	`((standard_hours * total_size_factor) - (contact_hours * approved_streams + other_load)) / contact_hours`	**Dual**
`qa.capacity.library.seat_ratio` / `material_ratio`	0.1 / 0.3333	Direct
`qa.capacity.lecturer_max_hours_week` / `admin_post_hours_week`	18 / 8	Direct
`qa.capacity.service_staff_ratio`	20	Direct

Key	Default	Control
`qa.capacity.rule_of_thumb`	`4 + 2 * (streams - 1)`	Direct
`qa.ratio.staff_student` / `qa.ratio.hnd_min_rank`	`students / counted_staff` / `lecturer-i`	Direct
`qa.capacity.default_contact_hours_week` / `default_room_seats`	20 / 40	Direct

Complaints

Key	Default	Control	Meaning
`qa.complaints.fallback_department`	`ict`	Direct (needs `complaints.manage_routing_rules`)	Queue for complaints no rule matches
`qa.complaints.sla_poll_minutes`	5	Direct	How often the SLA sweep runs
`qa.complaints.auto_close_days`	5	Direct	Days after resolution before auto-close
`qa.complaints.reopen_window_days`	14	Direct	Days in which the raiser may reopen
`qa.complaints.show_officer_name_to_raiser`	false	Direct	Show the handler's name to the raiser
`qa.complaints.sms_for_critical`	false	Direct	SMS when a critical complaint escalates (no SMS provider yet)

Interaction and backups

Key	Default	Control	Meaning
`qa.ui.max_show_all_rows`	500	Direct	Above this, *Show all* is disabled
`qa.ui.request_timeout_seconds`	15	Direct	Browser timeout before the retry message
`qa.backup.interval_days` / `keep_count` / `drill_every_n_runs` / `encrypt_at_rest`	30 / 8 / 4 / true	Direct (needs `backups.manage_destinations`; edited on Settings → Backups)	Backup schedule

---

9. Common mistakes

Mistake	What goes wrong	Do this instead
Changing a fee formula without testing	Bills go out at the wrong amount and cannot be changed afterwards (the amount is frozen)	Run the Test box for small, typical and large cases and each ownership before sending
Writing `'Private'` or `"federal "` in a formula	Text comparison is exact, so the condition is never true and everyone pays the other rate	Use the lookup **codes** exactly: `'private'`, `'federal'`, `'state'`
Forgetting that `cfg('…')` amounts are separate settings	The formula is right but the extra-stream charge is still ₦0	Check the *settings read* line in the test; propose the amount too
Not ticking a variable when adding a fee	*Unknown name 'streams'*	Tick every variable the formula uses
Renaming a lookup **code**	Fee formulas, document rules, routing conditions and imports stop matching	Change names freely; untick **active** to retire a value; change codes only with the developers
Expecting a deadline change to move open tasks	Tasks already open keep their due dates	Expect the change from the next task; ask the case owner to reassign or extend if needed
Asking the Director of Finance to confirm a fee **formula** started in DPAP	*This change belongs to another department*	A DPAP Director confirms settings started in DPAP; or the change is started inside Finance by someone with `edit_qa_settings`
Trying to confirm your own change	*You started this change, so a different officer must confirm it*	Ask another eligible officer
Starting a second change on the same item	*PA-… is already waiting for confirmation on this item*	Withdraw the first, or wait for a decision
Leaving a pending item too long	It expires after 72 hours and nothing is applied	Tell the confirmer when you send it; start again if it expired
Switching off a reference format	New records of that kind cannot be created (*No active reference format*)	Change the pattern instead; never untick **active** on a format in use
Changing a reference format's **scope** without a matching placeholder	A new counter starts at 1 and can repeat numbers already issued (for example per-department numbering without `{DEPT}`)	Include `{DEPT}` or `{INST}` when numbering per department or institution, and keep `{YEAR}` with yearly reset
A sensible-looking but wrong value	The system checks the *type*, not the *meaning*: `[150]` reminders or a department code that does not exist are accepted	Check the meaning; for department codes use the codes shown in Lookups and Departments

Mistake	What goes wrong	Do this instead
Publishing an SSQ version without preview	Institutions see confusing or missing questions on every new SSQ	Preview; have the Director review before **Publish**
Revising a catalogue line while institutions are filling SSQs	They may need to re-enter that line's quantity	Revise between QA request cycles where possible; tell affected institutions
A misspelt placeholder in a template	The value is silently left blank	Watch the live preview for gaps
Turning self-registration off without contact details	Visitors see only the site's general contact	Set the registration e-mail and phone first, then switch off
Testing notification changes on the live system	Real institutions receive test messages; the outbox stays empty	Test on a training copy and read the outbox
Expecting a workflow edit on screen	The Workflows tab is read-only in this release	Use the settings listed in 3.14 or request a new version (W10)

---

10. Troubleshooting

Symptom	Likely cause	What to do
403 — You do not have access to this QA screen	No settings permission (1.2)	Ask ICT to add you to the *QA Settings Admin* group or the right role
A section shows values but no **Edit** button	You can view but not change it; the row shows *You cannot edit this: Needs …*	Ask for the named permission
The rule shows *Fixed by policy*	The value is fixed in code	It needs a code release; it cannot be changed on screen
The new value is the same as the current one	Nothing to change	Check the value
Value is not a valid JSON (list/object)	A list or object is malformed	Use double quotes and square or curly brackets, e.g. `[50, 90]`, `["pdf", "png"]`
Formula rejected: Unknown name …	The formula uses a name that is not a declared variable	Use only the variables on the card; check spelling
Formula rejected: Syntax error	Brackets or quotes do not match	Count brackets; text in single quotes
cfg('…'): no such setting	A misspelt key inside `cfg()`	Copy the key from Rules & thresholds

Symptom	Likely cause	What to do
cfg() cycle: a → b → a	Two formulas read each other	Remove one of the `cfg()` references
Powers are limited to whole exponents of at most 4	`**` with a large or fractional exponent	Rewrite with multiplication
The formula gives zero or less for these inputs	A new fee or bill would be ₦0 or negative	Check the formula and sample inputs; a bill must be more than zero
Formula change *Sent for confirmation* but bills still use the old amount	It has not been confirmed yet	Ask the confirmer; the old formula stays in force until then
Confirmer cannot see the pending item	Different department or no confirm permission	Check the confirmer is in the initiator's department, holds the confirm permission and rank, or has *confirm any department*
This item has expired	Nobody decided in time	Start the change again
Stale — the item changed after this was started	The record changed after the proposal	Start the change again from the current value
A change saved but another screen still shows the old value	Development cache (up to 5 minutes in another process)	Wait five minutes or restart the worker; on production (Redis) it is immediate
Unconfirmed — bills blocked on the QA request fee	`qa.fee.qa_request_processing.confirmed` is false	When NBTE confirms the amounts, set the base and per-programme amounts and switch the confirmation on (all dual control)
Bills stay at *issued*, no slip	No real bank account is confirmed	Finance proposes the account; the Director of Finance confirms; waiting bills publish automatically
A payment method is on but not offered	Its server keys are missing	ICT sets the keys in `.env` and restarts; the Payment methods page names the missing keys
A new fee is never billed automatically	Nothing points at it yet	Issue it by hand (W3, Part 3) or request a workflow/QA type change
Version n is already a draft — edit or discard it first	An SSQ draft already exists	Open and finish that draft, or discard it
The template has errors when publishing an SSQ	The schema is invalid	Fix the listed problems; check *The schema is valid*
This draft is in use	SSQs were opened on the draft	It cannot be discarded; publish or keep it

Symptom	Likely cause	What to do
Catalogue *Nothing changed*	Same quantity, unit and name	Change at least one
Workload save says *Row n: … must be a whole number, or blank*	Text or a decimal in a cell	Enter whole numbers only
Workload save refused for a duplicate	The same rank appears twice	Keep one row per rank
Someone is refused for a visit or review	A workload limit applies (the message names it)	Check their load in the finder; adjust the limit only if NBTE policy changes
A complaint goes to ICT instead of the right department	No routing rule matched, or the rule's target found nobody	Check the rule order, the condition and that the target department has handlers
A routing condition is refused on save	It uses an unknown variable or does not give yes/no	Use the variables listed in 3.13 and a comparison such as `priority == "high"`
An escalation never happens	No active rule for that flow/stage, or *Only if* excludes it	Check Complaints → Escalation
A notification is not received	The template or rule is off, or the event has no recipient	Check the template is on; on a training copy look in the outbox
A notification shows a blank where a value should be	Misspelt or unavailable placeholder	Use the chips; check the live preview
Template error: … on save	Broken `{% %}` or `{{ }}` syntax	Close every `{% if %}` with `{% endif %}`; use `{{ name }}`
The outbox is empty on the live system	Live e-mail is sent for real, not stored	Expected; use a training copy to read e-mails
Unknown placeholder(s): {…} on a reference format	A placeholder not in the list	Use only `{YEAR}`, `{INST}`, `{DEPT}`, `{SEQ:n}`, `{CHK97}`, `{CHK11}`, `{CHK}`
The pattern must contain exactly one {SEQ}	No or two running numbers	Keep exactly one `{SEQ:n}`
You cannot change settings while impersonating another user	You are viewing as someone else	End the impersonation first
Registration still showing after switching off	Old page in the browser	Reload the portal page; the change is immediate

Appendix A. Addresses at a glance

Page	Address	URL name
Rules & thresholds	`/qa/settings/rules/`	`qa:settings_rules`
Formulas	`/qa/settings/formulas/`	`qa:settings_formulas`
Reference formats	`/qa/settings/references/`	`qa:settings_references`
Fees & bank accounts	`/qa/settings/fees/`	`qa_set:fees`
Add a fee (form)	`/qa/settings/fees/new/`	`qa_set:fee_new`
Payment methods	`/qa/settings/payment-methods/`	`qa_set:methods`
Rate table	`/qa/settings/rates/`	`qa_set:rates`
Document rules	`/qa/settings/documents/`	`qa_set:documents`
Lookups	`/qa/settings/lookups/<list>/`	`qa_set:lookups_kind`
SSQ templates	`/qa/settings/ssq-templates/`	`qa_set:ssq_templates`
Curriculum catalogue	`/qa/settings/catalogue/`	`qa_set:catalogue`
Approval types	`/qa/settings/approval-types/`	`qa_set:approval_types`
Workload limits	`/qa/settings/workload/`	`qa_set:workload`
Complaints	`/qa/settings/complaints/?section=layers` (or `routing`, `escalation`, `kb`)	`qa_set:complaints`
Workflows	`/qa/settings/workflows/`	`qa:settings_workflows`
Notification templates	`/qa/settings/templates/`	`qa:settings_templates`
Pending confirmations	`/qa/pending/`	`qa:pending_list`
E-mail outbox	`/qa/outbox/`	`qa:outbox`
Issue a bill	`/qa/finance/obligations/issue/`	`qa_fin:issue_bill`

Appendix B. Where things live in the code

Topic	Files
Settings catalogue (keys, defaults, controls)	`qa_core/settings_catalogue.py`
Settings write path	`qa_core/settings_service.py`, `config/models.py`, `config/registry.py`

Topic	Files
Settings screens (rules, formulas, references, workflows, templates, pending, outbox)	`qa_core/views.py`, `templates/qa_core/settings/`
Formula language	`qa_core/formula.py`
Two-step control	`qa_core/actions.py`, `qa_core.ActionPolicy`, `qa_core.PendingAction`
Reference numbers	`qa_core/references.py`
Notifications and e-mail	`qa_core/notify.py`, `qa_core/mail.py`
Fees, bank accounts, bills	`payments/views.py`, `payments/fee_admin.py`, `payments/handlers.py`, `payments/services.py`
SSQ templates, catalogue, rates, approval types	`accreditation/settings_views.py`, `accreditation/builder.py`, `accreditation/ssq.py`
Workload limits	`inspection/settings_views.py`, `inspection/workload.py`
Complaints settings	`complaints/settings_views.py`, `complaints/services.py`
Document rules and lookups	`institutions/views.py`, `institutions/forms.py`, `institutions/services.py`
Workflows	`qa_core/flow.py`, `qa_core/models.py` (`FlowDefinition`, `FlowStage`, `FlowTransition`, `FlowInstance`)
Seed data (roles, policies, flows, templates)	`qa_core/seed_data*.py`, `qa_core/management/commands/seed_startup.py`